



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ
НАУКА У НОВОМ САДУ



Проф. др Игор Дејановић

Шаблон и упутство за писање завршних радова

Нови Сад, 2026

	УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА 21000 НОВИ САД, Трг Доситеја Обрадовића 6	Број:
	ЗАДАТАК ЗА ЗАВРШНИ РАД	Датум:

(Податке уноси предметни наставник - ментор)

Студијски програм:	Софтверско инжењерство и информационе технологије		
Студент:	Уписати име и презиме	Број индекса:	Уписати индекс
Степен и врста студија:	Мастер академске студије		
Област:	Електротехничко и рачунарско инжењерство		
Ментор:	Игор Дејановић		
НА ОСНОВУ ПОДНЕТЕ ПРИЈАВЕ, ПРИЛОЖЕНЕ ДОКУМЕНТАЦИЈЕ И ОДРЕДБИ СТАТУТА ФАКУЛТЕТА ИЗДАЈЕ СЕ ЗАДАТАК ЗА ЗАВРШНИ РАД, СА СЛЕДЕЋИМ ЕЛЕМЕНТИМА: - проблем – тема рада; - начин решавања проблема и начин практичне провере резултата рада, ако је таква провера неопходна;			

НАСЛОВ ЗАВРШНОГ РАДА:

Шаблон и упутство за писање завршних радова

ТЕКСТ ЗАДАТКА:

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim aenean sit amet, adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim aenean sit amet, adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
--

Руководилац студијског програма:	Ментор рада:

Примерак за: ☐ - Студента; ☐ - Ментора
--

	УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА 21000 НОВИ САД, Трг Доситеја Обрадовића 6	
	КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА	

Редни број, РБР:		
Идентификациони број, ИБР:		
Тип документације, ТД:		Монографска документација
Тип записа, ТЗ:		Текстуални штампани материјал
Врста рада, ВР:		Дипломски - мастер рад
Аутор, АУ:		Уписати име и презиме
Ментор, МН:		Др Игор Дејановић, редовни професор
Наслов рада, НР:		Шаблон и упутство за писање завршних радова
Језик публикације, ЈП:		српски/хирилица
Језик извода, ЈИ:		српски/енглески
Земља публиковања, ЗП:		Република Србија
Уже географско подручје, УГП:		Војводина
Година, ГО:		2026
Издавач, ИЗ:		Ауторски репринт
Место и адреса, МА:		Нови Сад, трг Доситеја Обрадовића 6
Физички опис рада, ФО: (поглавља/страна/ цитата/табела/слика/графика/прилога)		10/67/10/8/17/0/2
Научна област, НО:		Електротехничко и рачунарско инжењерство
Научна дисциплина, НД:		Примењене рачунарске науке и информатика
Предметна одредница/Кључне речи, ПО:		Шаблон, завршни рад, упутство
УДК		
Чува се, ЧУ:		У библиотеци Факултета техничких наука, Нови Сад
Важна напомена, ВН:		
Извод, ИЗ:		Овај документ представља упутство за писање завршних радова на Факултету техничких наука Универзитета у Новом Саду. У исто време је и шаблон за Турpst.
Датум прихватања теме, ДП:		
Датум одбране, ДО:		01.01.2025
Чланови комисије, КО:	Председник:	Др Петар Петровић, ванредни професор
	Члан:	Др Марко Марковић, доцент
	Члан:	
	Члан, ментор:	Др Игор Дејановић, редовни професор
		Потпис ментора

	UNIVERSITY OF NOVI SAD • FACULTY OF TECHNICAL SCIENCES 21000 NOVI SAD, Trg Dositeja Obradovića 6	
	KEY WORDS DOCUMENTATION	
Accession number, ANO :		
Identification number, INO :		
Document type, DT : Monographic publication		
Type of record, TR : Textual printed material		
Contents code, CC :		
Author, AU : Upisati ime i prezime na latinici		
Mentor, MN : Igor Dejanović, Phd., full professor		
Title, TI : Template and tutorial for thesis preparation		
Language of text, LT : Serbian		
Language of abstract, LA : Serbian/English		
Country of publication, CP : Republic of Serbia		
Locality of publication, LP : Vojvodina		
Publication year, PY : 2026		
Publisher, PB : Author's reprint		
Publication place, PP : Novi Sad, Dositeja Obradovica sq. 6		
Physical description, PD : (chapters/pages/ref./tables/pictures/graphs/appendixes) 10/67/10/8/17/0/2		
Scientific field, SF : Electrical and Computer Engineering		
Scientific discipline, SD : Applied computer science and informatics		
Subject/Key words, S/KW : Template, thesis, tutorial		
UC		
Holding data, HD : The Library of Faculty of Technical Sciences, Novi Sad		
Note, N :		
Abstract, AB : This document provides guidelines for writing final theses at the Faculty of Technical Sciences, University of Novi Sad. At the same time, it serves as a Typst template.		
Accepted by the Scientific Board on, ASB :		
Defended on, DE : 01.01.2025		
Defended Board, DB :	President:	Petar Petrović, Phd., assoc. professor
	Member:	Marko Marković, Phd., asist. professor
	Member:	Mentor's sign
	Member, Mentor:	Igor Dejanović, Phd., full professor



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

ИЗЈАВА О НЕПОСТОЈАЊУ СУКОБА ИНТЕРЕСА

Изјављујем да нисам у сукобу интереса у односу ментор – кандидат и да нисам члан породице (супружник или ванбрачни партнер, родитељ или усвојитељ, дете или усвојеник), повезано лице (крвни сродник ментора/кандидата у правој линији, односно у побочној линији закључно са другим степеном сродства, као ни физичко лице које се према другим основама и околностима може оправдано сматрати интересно повезаним са ментором или кандидатом), односно да нисам зависан/на од ментора/кандидата, да не постоје околности које би могле да утичу на моју непристрасност, нити да стичем било какве користи или погодности за себе или друго лице било позитивним или негативним исходом, као и да немам приватни интерес који утиче, може да утиче или изгледа као да утиче на однос ментор-кандидат.

У Новом Саду, дана _____

Ментор

Кандидат

Садржај

1	Увод	1
1.1	Мотивација	2
1.2	Предмет и циљ рада	2
1.3	Основне функционалности алата RustyJudge	3
1.4	Организација рада	3
2	Статичка анализа и алати за проверу JavaScript кода	5
2.1	Улога linter-а	5
2.2	Постојећи алати	6
2.3	Нивои информација у анализи	6
2.4	Положај RustyJudge-а	7
3	Теоријске основе	9
3.1	Представљање изворног кода	9
3.2	Апстрактно синтаксичко стабло	10
3.3	Идентификатори, симболи и области важења	11
3.4	Контролно-проточни граф	12
3.5	Достижност	13
3.6	Анализа тока података	14
3.7	Анализа живости	15
3.8	Дијагностике и позиције у изворном коду	17
3.9	Значај теоријских представа за даљу имплементацију	18
4	Архитектура алата RustyJudge	19
4.1	Организација пројекта	19
4.2	Главни ток анализе	20
4.3	Улазне тачке и заједничко језгро	21
4.4	CLI режим	22
4.5	Језгро linter-а и контекст правила	23
4.6	Семантички модел као унутрашњи слој анализе	23
4.7	Дијагностике и излазни формати	24
4.8	LSP сервер и VS Code проширење	24
4.9	Benchmark интеграција	25
4.10	Архитектонске последице	25
5	Имплементација контролно-проточног графа	27
5.1	Место CFG модула у пројекту	27
5.2	Основне структуре података	28
5.3	Улога CfgBuilder компоненте	29

5.4	Спуштање синтаксичког стабла	30
5.5	Декларације, доделе и употребе симбола	30
5.6	Условне наредбе	31
5.7	Петље и циљеви за break и continue	31
5.8	Функције као посебна тела анализе	32
5.9	try/catch модел	33
5.10	Завршавање и чишћење графа	33
5.11	Тренутни обим подршке	33
6	Имплементација анализе живости података	35
6.1	Место анализе у семантичком моделу	35
6.2	Структура LivenessResult	36
6.3	Достижност као први корак	37
6.4	Формирање use и def скупова	38
6.5	Израчунавање блоковске живости	38
6.6	Живост на нивоу исказа	39
6.7	Пример мртве доделе	39
6.8	Ограничења тренутног модела	40
7	Имплементација правила	41
7.1	Преглед имплементираних правила	41
7.2	Правила над текстом и синтаксом	42
7.3	Правила над симболима и достижношћу	43
7.4	Правила над повратним путањама	44
7.5	Правило за мртве доделе	44
7.6	Правило за бесконачне петље	45
7.7	Дијагностике и тестови	45
8	CLI и VS Code интеграција	47
8.1	CLI режим	47
8.2	LSP сервер	48
8.3	VS Code проширење	48
9	Мерење перформанси	51
10	Закључак	53
	Списак слика	55
	Списак листинга	57
	Списак табела	59
	Списак коришћених скраћеница	61
	Списак коришћених појмова	63
	Биографија	65
	Литература	67

Глава 1

Увод

Савремени софтверски системи се у великој мери ослањају на језике и алате који омогућавају брз развој, једноставну интеграцију и извршавање на различитим платформама. Један од најзначајнијих језика у том контексту је JavaScript, који се користи у развоју веб апликација, серверских система, алата командне линије, десктоп апликација и проширења за развојна окружења. Због широке примене, JavaScript код често настаје у великим и дуговечним пројектима, у којима га мења више програмера и у којима грешке у коду могу да имају значајан утицај на поузданост, одрживост и разумљивост система.

Флексибилност JavaScript језика истовремено представља и предност и изазов. Језик дозвољава различите стилове програмирања, динамичку природу вредности, слободнију употребу променљивих и велики број синтаксичких конструкција. Таква флексибилност убрзава развој, али може да отежа уочавање грешака пре извршавања програма. Неке неправилности су синтаксичке и лако се откривају већ током парсирања кода. Међутим, велики број проблема није видљив само из појединачне синтаксичке конструкције. За њихово откривање потребно је разумети односе између наредби, ток извршавања програма, области важења идентификатора и начин на који се вредности дефинишу и користе.

Један од приступа за рано откривање таквих проблема је статичка анализа програмског кода. Статичка анализа обухвата поступке у којима се програм испитује без његовог покретања. У пракси се ова идеја често примењује кроз алате познате као linter-и. Linter-и анализирају изворни код и пријављују потенцијалне грешке, сумњиве обрасце, некоришћене делове програма, непожељан стил писања или конструкције које могу да доведу до тежег одржавања. Посебно су корисни када се укључе у свакодневни ток рада, јер програмеру дају повратну информацију пре него што грешка стигне до тестирања или продукционог окружења.

У овом раду приказан је развој алата RustyJudge, linter-a за JavaScript написаног у програмском језику Rust. Алат је осмишљен као статички анализатор који, поред основне синтаксичке обраде, гради унутрашње представе програма потребне за сложенија правила провере. Посебна пажња посвећена је изградњи контролно-точног графа (CFG, енг. *Control-Flow Graph*), анализи достижности делова програма и анализи живости података (енг. *liveness analysis*). На основу тих информација мо-

гуће је имплементирати правила која не зависе само од облика једне синтаксичке конструкције, већ и од начина на који се програм може извршавати.

1.1 Мотивација

Основна мотивација за развој алата RustyJudge произилази из потребе да се поступак статичке анализе JavaScript кода прикаже као целовит процес. Уместо да се linter посматра само као листа правила, у раду се прати пут од изворног кода до дијагностике: парсирање, изградња унутрашњих структура, покретање правила и приказ резултата кориснику. Такав приказ омогућава да се јасно види које информације су потребне за одређену врсту провере.

Програмски језик Rust изабран је због карактеристика које су погодне за развој системских и развојних алата. Он омогућава високе перформансе, експлицитно управљање подацима и строжију контролу над исправношћу програма у време компајлирања. За алат који обрађује изворни код, гради графове и покреће више правила анализе, ове особине су значајне јер утичу на брзину извршавања, организацију података и поузданост имплементације.

1.2 Предмет и циљ рада

Предмет рада је пројектовање и имплементација алата RustyJudge за статичку анализу JavaScript кода. Алат чита изворне датотеке, парсира их, гради унутрашње структуре потребне за анализу, покреће скуп правила и кориснику приказује дијагностике. Дијагностика представља поруку о уоченом проблему, заједно са позицијом у изворном коду и описом правила које је прекршено.

Циљ рада је да се прикаже како се један linter може изградити као компајлерски оријентисан алат. То подразумева да се имплементација не заснива само на директном обиласку синтаксичког стабла, већ и на изградњи богатијих модела програма. Посебни циљеви рада су:

- приказ поступка парсирања JavaScript кода и добијања AST структуре;
- изградња семантичког контекста са информацијама о симболима и областима важења;
- изградња CFG представе погодне за анализу тока извршавања;
- имплементација анализе достижности и анализе живости података;
- дефинисање инфраструктуре за извршавање правила;
- имплементација конкретних правила linter-a;
- приказ дијагностика кроз CLI, LSP и VS Code проширење;
- поређење перформанси са алатима ESLint и RSLint.

Овако постављен циљ омогућава да се RustyJudge посматра из два угла. Са једне стране, он је практичан алат који може да анализира JavaScript датотеке и прикаже

кориснику проблеме у коду. Са друге стране, он је пример примене концепата из области компајлера и статичке анализе на конкретан програмски језик и конкретан развојни алат.

1.3 Основне функционалности алата RustyJudge

RustyJudge је организован као Rust пројекат који садржи библиотечки део, извршне улазе за CLI и LSP, скуп правила, модуле за CFG и анализу података, benchmark окружење и VS Code проширење. Основни ток рада почиње читањем JavaScript изворног кода и његовим парсирањем. Након тога се над добијеном структуром покрећу анализе и правила која производе дијагностике.

Унутар алата раздвојене су фазе анализе. Један део система задужен је за парсирање и обилазак синтаксичке структуре. Други део гради семантички контекст са информацијама о симболима и областима важења. Трећи део формира CFG и податке потребне за анализу живости. Над тим структурама покрећу се правила која производе дијагностике. Оваква подела омогућава да се свако правило ослони на онај ниво информација који му је стварно потребан.

Поред анализе у командној линији, RustyJudge садржи и интеграцију са развојним окружењем. LSP сервер омогућава да се резултати анализе прикажу у едитору, док VS Code проширење служи као клијентска страна која повезује корисника, отворену датотеку и RustyJudge анализатор. На тај начин се дијагностике могу приказати директно на месту где програмер пише код, што је природан начин употребе linter-a у свакодневном развоју.

Значајан део рада представља и benchmark анализа. Пошто постоје већ познати алати за анализу JavaScript кода, RustyJudge је поређен са постојећим решењима. Ово поређење не служи само да покаже време извршавања, већ и да се објасни контекст мерења: које су датотеке анализиране, која правила су покренута, како су алати конфигурисани и која су ограничења таквог поређења.

1.4 Организација рада

Рад је организован тако да се од општих појмова постепено прелази ка конкретној имплементацији алата RustyJudge. У другом поглављу приказани су појам статичке анализе, улога linter-a у развоју софтвера и постојећи алати за анализу JavaScript кода, са посебним освртом на ESLint и RSLint.

У трећем поглављу изложене су теоријске основе потребне за разумевање имплементације. Описани су AST, области важења, CFG, основни блокови, достижност, ток података и анализа живости. Ово поглавље поставља основу за касније разумевање начина на који RustyJudge представља и анализира програм.

Четврто поглавље описује архитектуру RustyJudge-а. Приказан је ток података од улазне JavaScript датотеке до дијагностике, као и улога главних модула у систему. Пето поглавље посвећено је имплементацији CFG-а и начину на који се поједине JavaScript конструкције преводе у граф тока извршавања.

У шестом поглављу описана је анализа живости података и њена примена у оквиру алата. Седмо поглавље приказује инфраструктуру за правила linter-а и конкретна правила која су имплементирана. Осмо поглавље описује употребу алата кроз CLI, LSP и VS Code проширење.

Девето поглавље посвећено је евалуацији и поређењу перформанси са алатима ESLint и RSLint. У њему су приказани услови мерења, резултати и ограничења поређења. На крају, у закључку су сумирани резултати рада, наведена ограничења тренутне имплементације и предложени правци даљег развоја.

Глава 2

Статичка анализа и алати за проверу JavaScript кода

Статичка анализа је поступак испитивања програмског кода без његовог извршавања. Циљ такве анализе је да се из самог текста програма, његове синтаксичке структуре и додатних унутрашњих представа изведу закључци о могућим грешкама, недостижном коду, непотребним деловима програма или непожељним обрацима писања. У теорији програмских језика и компајлера ова област је позната као анализа програма (енг. *program analysis*) [1].

За разлику од динамичког тестирања, статичка анализа не захтева покретање програма нити припрему улазних података. Она зато може да се извршава рано у развоју, на пример приликом чувања датотеке, пре покретања тестова или у оквиру система за континуирану интеграцију. Са друге стране, статичка анализа има ограничења. Пошто се програм не извршава, резултат анализе зависи од модела који алат гради о програму. Једноставнији модел даје бржу анализу, али мање прецизне закључке. Богатији модел омогућава сложенија правила, али повећава сложеност имплементације.

У случају JavaScript језика, статичка анализа је посебно значајна јер се језик користи у различитим окружењима и подржава велики број програмских стилова. Званична спецификација језика дефинисана је стандардом ECMA-262 [2]. Међутим, практична провера кода у развојном окружењу не своди се само на проверу да ли је програм синтаксички исправан. Потребно је открити и код који је технички исправан, али указује на могућу грешку или слабију одрживост.

2.1 Улога linter-a

Linter је практичан облик статичког анализатора. Он чита изворни код, покреће скуп правила и враћа дијагностике. Дијагностика обично садржи место проблема, кратку поруку и ознаку правила које је пријавило проблем. У зависности од алата, дијагностика може да има различит ниво озбиљности, као што су грешка, упозорење или информација.

Вредност linter-a није само у томе што проналази проблеме, већ и у тренутку у коме их пријављује. Ако се покреће у едитору, програмер добија повратну информацију док пише код. Ако се покреће у CLI окружењу, исти скуп правила може да се

користи локално, у скриптама или у систему за континуирану интеграцију. На тај начин linter постаје део развојног процеса, а не само алат који се покреће на крају рада.

Квалитет linter-a зависи од неколико особина. Дијагностике треба да буду довољно прецизне да програмер брзо пронађе узрок проблема. Број лажно позитивних пријава мора бити низак, јер се у супротном упозорења игноришу. Анализа мора бити довољно брза да не прекида рад у едитору. Правила треба да буду јасно именована и, када је могуће, подесива. Ове особине су практичне, али директно утичу на архитектуру алата: подаци морају бити организовани тако да правила имају довољно информација, али да анализа остане брза.

2.2 Постојећи алати

Најпознатији linter у JavaScript екосистему је ESLint. Он је проширив алат који се заснива на правилима, конфигурацији и могућности додавања plugin-a. Документација ESLint-a правила описује као основне градивне елементе алата: свако правило проверава да ли код испуњава одређено очекивање и пријављује проблем када то није случај [3]. Због великог броја уграђених правила и plugin-a, ESLint је у пракси чест избор за JavaScript и TypeScript пројекте.

ESLint је други значајан алат у контексту овог рада. У питању је JavaScript linter написан у Rust-у, са нагласком на брзину, мању потрошњу меморије и практичан CLI рад [4]. За RustyJudge је ESLint посебно релевантан зато што омогућава поређење са алатом из сличног технолошког простора. У benchmark делу рада ESLint се користи кроз CLI, па се мери укупно време извршавања, без раздвајања унутрашњих фаза као што су парсирање и извршавање правила.

Табела 1: Поређење алата релевантних за рад.

Алат	Разлог за поређење
ESLint	Представља широко коришћено решење из праксе и добар оријентир за понашање JavaScript linter-a.
ESLint	Омогућава поређење са алатом који је технолошки ближи RustyJudge-у.
RustyJudge	Предмет је имплементације и омогућава мерење појединачних фаза анализе.

Табела 1 приказује улогу сваког алата у овом раду.

2.3 Нивои информација у анализи

Правила linter-a могу да се посматрају кроз ниво информација који им је потребан. Не захтева свако правило исту дубину анализе. Нека правила могу да се изврше над

појединачним чвором синтаксичког стабла, док друга морају да користе области важења, ток извршавања или ток података. Ова разлика је важна јер утиче на цену анализе и на организацију имплементације.

Табела 2: Нивои информација потребни за различите врсте lint правила.

Ниво анализе	Информације	Пример провере
Синтаксички	Облик израза, наредби и декларација.	Препознавање непожељне синтаксичке конструкције.
Семантички	Веза између употребе имена и његове декларације.	Откривање некоришћене декларације или погрешне употребе идентификатора.
Контролно-проточни	Могуће путање извршавања програма.	Откривање недостижног кода после наредбе која прекида ток.
Ток података	Места на којима се вредност дефинише, користи или престаје да буде потребна.	Откривање доделе чији резултат касније није употребљен.

Табела 2 показује да избор унутрашње представе није само технички детаљ, већ одређује које врсте правила алат може природно да подржи. Зато је корисно да linter има јасно раздвојене слојеве анализе. Једноставна правила не треба да плаћају цену сложених анализа, док сложенија правила морају имати приступ подацима који превазилазе непосредни облик кода.

2.4 Положај RustyJudge-a

RustyJudge није замишљен као потпуна замена за ESLint. ESLint има велики број правила, развијен plugin екосистем и дугу примену у реалним пројектима. RustyJudge има ужи циљ: да прикаже како се linter може изградити око јасно одвојених фаза анализе и како се те фазе могу искористити за правила која зависе од тока извршавања и тока података.

У односу на постојеће алате, вредност RustyJudge-a у овом раду није у броју подржаних правила, већ у транспарентности архитектуре. Фазе анализе су раздвојене тако да се може мерити време парсирања, изградње семантичког модела и CFG-a, извршавања правила и укупног linting процеса. То је важно за benchmark поглавље, јер омогућава да се резултати не посматрају само као један број, већ као збир више корака.

Овакво позиционирање одређује и остатак рада. Наредно поглавље зато не улази одмах у имплементацију, већ уводи појмове који су потребни да би се она разумела: AST, области важења, CFG, достижност, ток података и анализа живости. Након тога се приказује како су ти појмови реализовани у RustyJudge-у.

Глава 3

Теоријске основе

Ово поглавље уводи појмове који су потребни за разумевање касније имплементације алата RustyJudge. Нагласак није на конкретном коду алата, већ на моделима који се уобичајено користе у компајлерима, статичким анализаторима и развојним алатима. Изворни програм се најпре посматра као текст, затим као синтаксичка структура, потом као скуп симбола и области важења, а на крају као граф над којим се могу рачунати особине тока извршавања и тока података.

У наставку се користи један једноставан JavaScript пример. Исти пример служи за објашњење AST-а, CFG-а и анализе живости, како би се видело како се различите представе односе на исти део програма.

```
function max(a, b) {  
    let result = a;  
  
    if (b > result) {  
        result = b;  
    }  
  
    return result;  
}
```

Листинг 1: Једноставан JavaScript пример који се користи у овом поглављу.

Листинг 1 садржи функцију која враћа већу од две вредности. Иако је пример мали, у њему постоји више елемената важних за анализу: декларација променљиве, употреба параметара, условно гранање, додела вредности и повратак из функције. То је довољно да се прикажу основни појмови без увођења непотребне сложености.

3.1 Представљање изворног кода

Изворни код је на почетку само низ карактера. Да би програмски алат могао да га анализира, тај низ мора да се преведе у структуру погодну за обраду. Први корак је лексичка анализа. Њен задатак је да препозна токене, односно најмање синтаксички значајне јединице програма. У примеру из листинга 1 токени су, између осталог, кључна реч `function`, идентификатор `max`, заграде, зарези, кључна реч `let`, оператор `>`, оператор доделе `=` и идентификатори `a`, `b` и `result`.

Токен садржи најмање две врсте информација. Прва је врста токена, на пример идентификатор, број, кључна реч или оператор. Друга је положај у изворном коду.

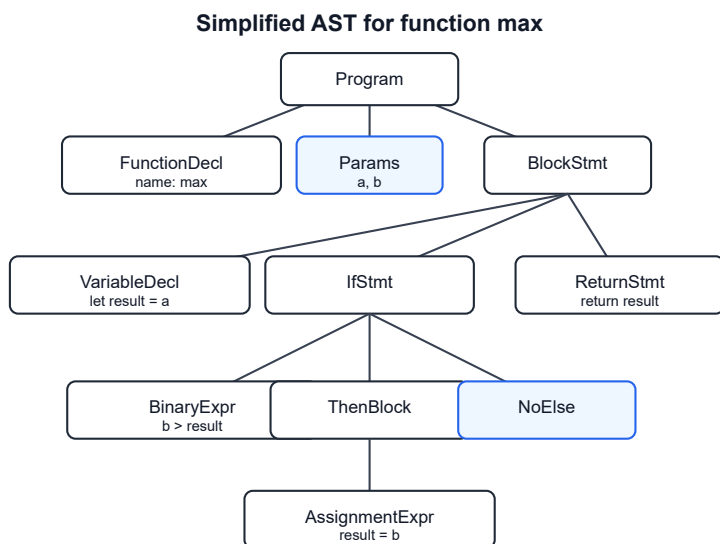
Положај је неопходан зато што се касније дијагностике морају везати за тачно место у датотеци. Без те информације алат би могао да закључи да проблем постоји, али не би могао кориснику да покаже где се налази.

После лексичке анализе следи синтаксичка анализа, односно парсирање. Парсер проверава да ли редослед токена одговара граматички језика и гради структуру која представља програм. У компајлерској литератури ове фазе се описују као предњи део компајлера, јер преводе текст програма у облик над којим касније фазе могу да раде [5].

Важно је разликовати конкретно синтаксичко стабло и апстрактно синтаксичко стабло. Конкретно синтаксичко стабло задржава велики број детаља о граматички, као што су помоћни нетерминали, интерпункција и тачан облик правила. AST изоставља детаље који нису потребни за каснију анализу и задржава суштинску структуру програма. У пракси алати често користе структуре које се налазе између ова два облика, јер је за дијагностике корисно сачувати и део информација о изворном тексту.

3.2 Апстрактно синтаксичко стабло

AST је стабло у коме чворови представљају програмске конструкције. Корен обично представља целу датотеку или програм. Унутрашњи чворови представљају конструкције као што су декларације, функције, блокови, наредбе и изрази, док листови садрже једноставне елементе као што су идентификатори, литерали и оператори.



Слика 1: Поједностављени AST за функцију из листинга 1.

Слика 1 приказује поједностављени AST за функцију `max`. У стварној имплементацији стабло може да садржи више детаља, али су основни односи исти. Функција има име, листу параметара и тело. Тело садржи декларацију променљиве, условну наредбу и `return` наредбу. Условна наредба садржи израз који се проверава и блок који се извршава ако је услов тачан.

AST је погодан за многе облике анализе јер чува хијерархију програма. Ако се анализира функција, лако је пронаћи њене параметре и тело. Ако се анализира условна наредба, лако је пронаћи услов и гране. Ако се анализира декларација, чвор стабла обично садржи име идентификатора и почетну вредност. Због тога је AST природна полазна тачка за `linter` правила, компајлерске трансформације и алате за форматирање кода.

Међутим, AST не треба посматрати као једину представу програма. Он описује утнежђеност синтаксичких конструкција, али не описује експлицитно све односе који су важни за касније фазе. На пример, појава идентификатора `result` у изразу не говори сама по себи која декларација уводи тај идентификатор. Та веза се успоставља семантичком анализом. Слично томе, редослед чворова у стаблу није увек исто што и редослед могућег извршавања, па се за ту сврху користи CFG.

3.3 Идентификатори, симболи и области важења

Идентификатор је име које се појављује у програму. У JavaScript-у идентификатор може да означава променљиву, параметар, функцију, класу или друго именовано место у програму. Симбол је апстрактна представа тог именованог ентитета. Разлика између идентификатора и симбола је важна: исти текст имена може да се појави на више места, али не мора увек да означава исти симбол.

Област важења, или *scope*, одређује део програма у коме је одређени симбол видљив. JavaScript има више врста области важења. Параметри функције важе у телу функције. Променљиве декларисане са `let` и `const` имају блоковску област важења. Променљиве декларисане са `var` нису блоковски ограничене: ако су декларисане унутар функције, везују се за област те функције, док се на највишем нивоу везују за одговарајући глобални или модулски контекст. Ови детаљи су део семантике језика дефинисане ECMAScript спецификацијом [2].

Семантичка анализа обично гради структуру која личи на табелу симбола. У њој се бележи који симбол је уведен којом декларацијом, у којој области важи и на које се појаве идентификатора односи. За пример из листинга 1 могу се издвојити симболи `max`, `a`, `b` и `result`. Симбол `max` представља функцију. Симболи `a` и `b` представљају параметре функције. Симбол `result` представља локалну променљиву декларисану у телу функције.

Разрешавање имена је поступак повезивања употребе идентификатора са одговарајућом декларацијом. Када се у изразу `b > result` појави име `b`, анализа треба да га повеже са параметром `b`. Када се појави `result`, анализа треба да га повеже са локалном декларацијом `let result = a`. Ова веза је основа за правила која се баве употребом променљивих, поновним декларацијама, сенчењем имена и другим семантичким појавама.

Угнежђене области важења могу се посматрати као стабло. Унутрашња област може да приступи симболима из спољашње области ако их не заклања својом декларацијом. Ако се у унутрашњој области декларише нови симбол са истим именом, он може да засени симбол из спољашње области. Због тога алат не сме да посматра само текст идентификатора, већ мора да узме у обзир место на коме се идентификатор налази.

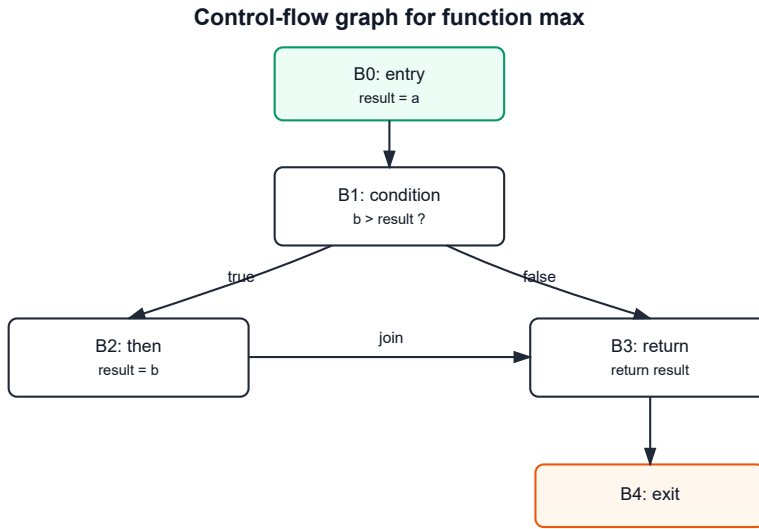
3.4 Контролно-проточни граф

Контролно-проточни граф је графичка и математичка представа могућег тока извршавања програма. Означава се као:

$$G = (V, E, \text{entry}, \text{exit})$$

У овој формули V представља скуп чворова, E скуп усмерених грана, `entry` улазни чвор, а `exit` излазни чвор графа. Чворови могу да представљају појединачне наредбе или основне блокове. Основни блок је низ наредби који има један улаз и један излаз, без унутрашњег гранања. Гране у графу показују куда извршавање може да пређе после неког блока.

Основни блокови се користе зато што поједностављују анализу. Уместо да се свака наредба посматра као посебан чвор, више узастопних наредби без гранања може да се групише у један блок. На крају блока обично се налази конструкција која одређује следећи корак: услов, безусловни прелаз, `return`, `throw`, крај функције или друга наредба која утиче на ток извршавања.



Слика 2: CFG за функцију из листинга 1.

Слика 2 приказује CFG за функцију `max`. Блок `B0` представља улаз у функцију и иницијализацију променљиве `result`. Блок `B1` представља проверу услова `b > result`. Ако је услов тачан, извршавање прелази у блок `B2`, где се променљивој `result` додељује вредност `b`. Ако услов није тачан, прелази се директно у блок `B3`, који враћа вредност `result`. После блока `B2` ток се такође спаја у блоку `B3`. На крају, извршавање прелази у излазни чвор `B4`.

Условне наредбе, петље и наредбе прекида тока стварају карактеристичне облике у CFG-у. Условна наредба има најмање две излазне гране: једну за случај када је услов тачан и једну за случај када није. Петља обично има повратну грану која враћа ток на проверу услова или на почетак тела петље. Наредба `return` води ка излазу из функције и после ње се у истом току не наставља извршавање наредби које следе у тексту програма.

CFG се користи у многим анализама јер програм претвара у структуру над којом се могу применити алгоритми над графовима. Могуће је израчунати који су блокови достижни, који блокови претходе неком блоку, који блокови следе после њега и како се информације преносе кроз програм. Ови појмови су основа за анализу тока података.

3.5 Достижност

Достижност је својство чвора у графу. Чвор је достижан ако постоји путања од улазног чвора до тог чвора. У контексту програма, недостижан блок представља део

кода до кога извршавање не може да стигне. Такав код је често последица грешке, застарелог дела програма или погрешно написане контролне структуре.

Достижност се може израчунати једноставним обиласком графа од улазног чвора. На почетку су сви чворови необележени. Улазни чвор се означава као достижан и ставља у радну листу. Док листа није празна, узима се један чвор и обилазе се његови наследници. Сваки наследник који још није означен додаје се у радну листу. Када се поступак заврши, сви необележени чворови су недостижни.

```
worklist = [entry]
reachable = {}

while worklist is not empty:
    block = remove_one(worklist)

    if block is already in reachable:
        continue

    add block to reachable

    for successor in successors(block):
        add successor to worklist
```

Листинг 2: Псеудокод алгоритма за израчунавање достижних блокова.

Листинг 2 приказује основну идеју алгоритма. У пракси се могу користити различити облици обиласка, на пример DFS или BFS, али резултат је исти: скуп блокова који се могу достићи из улазног чвора. Овај поступак је једноставан, али је веома користан у linter-има јер омогућава пријављивање делова програма који се не могу извршити.

3.6 Анализа тока података

Анализа тока података испитује како се информације мењају и преносе кроз CFG. За разлику од достижности, која рачуна само да ли је чвор могуће посетити, анализа тока података рачуна неку особину програма у сваком блоку. Та особина може да буде скуп живих променљивих, скуп доступних дефиниција, скуп израза који су већ израчунати или нека друга информација важна за анализу.

Општи облик анализе тока података може се описати помоћу улазног скупа, излазног скупа и функције преноса. За блок B користе се ознаке:

- $\text{in}(B)$ - информације које важе на улазу у блок;
- $\text{out}(B)$ - информације које важе на излазу из блока;
- transfer_B - функција која описује како блок мења информације.

У најопштијем облику важи:

$$\text{out}(B) = \text{transfer}_{B(\text{in}(B))}$$

Анализа може бити унапредна или уназадна. Унапредна анализа преноси информације од почетка програма ка крају. Пример такве анализе је анализа доступних дефиниција, где се питање поставља у облику: које дефиниције могу да стигну до ове тачке програма. Уназадна анализа преноси информације од краја програма ка почетку. Анализа живости је уназадна анализа, јер се питање поставља у облику: које вредности ће бити потребне у наставку извршавања.

Када граф садржи гранања и петље, један пролаз кроз блокове обично није довољан. Информације се морају рачунати итеративно, све док се не достигне фиксна тачка. Фиксна тачка је стање у коме нова итерација више не мења израчунате скупове. Пошто су скупови у овим анализама коначни, а функције преноса се дефинишу тако да монотono додају или уклањају информације у контролисаном облику, итеративни поступци су стандардни начин израчунавања резултата [1].

За спајање информација из више грана користи се оператор спајања. Код анализа које питају шта је могуће, често се користи унија скупова. Код анализа које питају шта сигурно важи на свим путањама, често се користи пресек. Избор оператора зависи од значења анализе.

3.7 Анализа живости

Променљива је жива у некој тачки програма ако њена тренутна вредност може бити употребљена на некој путањи извршавања која почиње у тој тачки. Ако вредност није жива, њено чување или додељивање може бити непотребно. Због тога је анализа живости важна у компајлерима, оптимизацијама и linter правилима.

Анализа живости користи два локална скупа за сваки блок:

- $\text{use}(B)$ - променљиве које се у блоку користе пре него што се у том блоку дефинишу;
- $\text{def}(B)$ - променљиве којима се у блоку додељује вредност или које се у блоку дефинишу.

Поред њих рачунају се и два глобална скупа:

- $\text{in}(B)$ - променљиве живе на улазу у блок;
- $\text{out}(B)$ - променљиве живе на излазу из блока.

Пошто је живост уназадна анализа, $\text{out}(B)$ зависи од улазних скупова наследника блока B :

$$\text{out}(B) = \bigcup_{S \in \text{succ}(B)} \text{in}(S)$$

Скуп $\text{in}(B)$ добија се тако што се узму променљиве које се користе у самом блоку и додају променљиве које су живе после блока, осим оних које се у блоку поново дефинишу:

$$\text{in}(B) = \text{use}(B) \cup (\text{out}(B) - \text{def}(B))$$

У овој формули $\text{succ}(B)$ означава наследнике блока B , а знак $-$ представља разлику скупова. Интуиција је следећа: ако се променљива користи у блоку пре нове дефиниције, она мора бити жива на улазу у блок. Ако је променљива жива на излазу из блока, она је жива и на улазу, осим ако блок пре тога дефинише нову вредност која замењује стару.

```
for each block B:
    in[B] = {}
    out[B] = {}

repeat:
    changed = false

    for each block B in reverse order:
        old_in = in[B]
        old_out = out[B]

        out[B] = union of in[S] for every successor S of B
        in[B] = use[B] union (out[B] - def[B])

        if in[B] != old_in or out[B] != old_out:
            changed = true
    until changed == false
```

Листинг 3: Псеудокод итеративне анализе живости.

Листинг 3 приказује типичан итеративни алгоритам. Блокови се често обилазе обрнутим редоследом у односу на ток извршавања, јер је анализа уназадна. Тај редослед није услов за исправност, али може да убрза достизање фиксне тачке.

Табела 3: Скупови анализе живости за CFG са слике 2.

Блок	Наредба	use	def	in	out
B0	result = a	{a}	{result}	{a, b}	{b, result}
B1	b > result	{b, result}	{}	{b, result}	{b, result}
B2	result = b	{b}	{result}	{b}	{result}
B3	return result	{result}	{}	{result}	{}
B4	exit	{}	{}	{}	{}

Табела 3 приказује резултат анализе за пример `max`. На улазу у блок B0 живи су параметри `a` и `b`. Вредност `a` је потребна за иницијализацију променљиве `result`, а вредност `b` је потребна касније у услову и могућој додели. На излазу из блока B0 жива је вредност `result`, али и параметар `b`, јер ће се оба користити у услову `b > result`. После блока B2 жива је само вредност `result`, јер се она враћа у блоку B3.

Овакав резултат показује како се локалне информације из појединачних блокова повезују са глобалном структуром графа. У блоку B2 променљива `result` се дефинише, па стара вредност `result` није жива на улазу у тај блок. Међутим, нова вредност јесте жива на излазу, јер се користи у `return` наредби. Управо ова врста разликовања старе и нове вредности чини анализу живости корисном за правила која испитују употребу резултата доделе.

3.8 Дијагностике и позиције у изворном коду

Резултат статичке анализе није користан ако се не може јасно приказати кориснику. Зато алати за анализу кода производе дијагностике. Дијагностика је структурирана порука која обично садржи:

- идентификатор правила;
- ниво озбиљности;
- поруку за корисника;
- позицију у изворном коду;
- опциони предлог исправке или додатно објашњење.

Позиција у изворном коду најчешће се представља као распон. Распон може бити задат помоћу почетног и крајњег индекса у датотеци, или помоћу броја реда и колоне. За приказ у едитору обично је потребно претворити интерне индексе у редове и колоне, јер корисник проблем види у том облику. Ако је анализа извршена над AST-ом или CFG-ом, сваки чвор који може да произведе дијагностику треба да задржи везу са делом изворног кода из кога је настао.

Добра дијагностика треба да буде кратка, али довољно прецизна. Порука не треба само да каже да је правило прекршено, већ треба да помогне програмеру да разуме зашто је место означено. Код linter-а је то посебно важно јер се дијагностике појављују током писања кода. Ако су поруке нејасне или ако означавају превелики део програма, корисник теже прихвата алат као део редовног рада.

3.9 Значај теоријских представа за даљу имплементацију

Појмови уведени у овом поглављу чине основу за наредна поглавља. AST описује структуру изворног кода. Символи и области важења повезују имена са њиховим значењем у програму. CFG описује могуће путање извршавања. Анализа тока података омогућава да се над CFG-ом израчунају особине које зависе од више блокова. Анализа живости је конкретан пример такве анализе и користи се за разумевање употребе вредности у програму.

Наредна поглавља показују како се ови појмови примењују у RustyJudge-у. Прво се описује архитектура алата и проток података кроз систем. Након тога се детаљније приказује изградња CFG-а, израчунавање живости и начин на који правила користе добијене информације да би произвела дијагностике.

Глава 4

Архитектура алата RustyJudge

Ово поглавље описује како је RustyJudge организован као софтверски систем. Претходно поглавље увело је појмове који се користе у статичкој анализи, док се овде посматрају конкретни модули, улазне тачке и ток података кроз имплементацију.

RustyJudge је реализован као Rust пројекат који садржи бибљотечки део, извршне програме, LSP сервер, benchmark окружење и VS Code проширење. Важна архитектонска одлука је да се главна логика анализе налази у заједничкој бибљотеци. На тај начин CLI, LSP сервер и benchmark код не садрже сопствене варијанте анализе, већ користе исти основни ток.

4.1 Организација пројекта

Изворни код Rust дела алата налази се у директоријуму `src/`. Модули су подељени према одговорностима. Улазни бинарни програми су мали и служе само да покрену одговарајући режим рада. Главни део понашања налази се у бибљотечким модулима који се могу поново користити.

Табела 4: Главни делови RustyJudge пројекта.

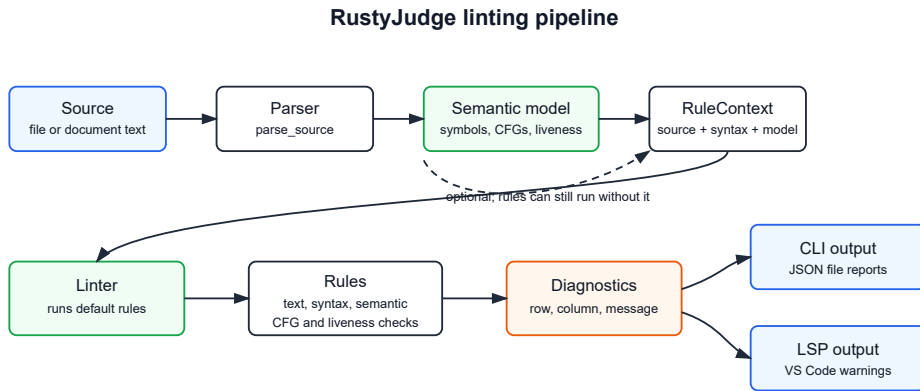
Модул или директоријум	Одговорност
src/parser/	Омотач око rslint_parser библиотеке, читање JavaScript извора и враћање синтаксичког корена или грешке парсирања.
src/cfg/	Семантичко спуштање, табела симбола, CFG структуре, изградња графа и анализа живости.
src/rules/	Интерфејс правила, RuleContext, подразумевани скуп правила и појединачне имплементације правила.
src/linter/	Језгро које прави контекст, покреће правила и враћа дијагностике.
src/diagnostics/	Заједнички формат дијагностике који користе CLI и LSP слој.
src/cli/ и src/run_cli/	Обрада аргумената командне линије, читање датотека и JSON приказ резултата.
src/lsp/	LSP сервер заснован на tower-lsp библиотеци и конверзија RustyJudge дијагностика у LSP дијагностике.
benchmarks/	Генерисање корпуса и мерење појединачних фаза анализе и укупног времена извршавања.
vscode-extension/	TypeScript клијент који покреће Rust LSP бинарну датотеку и повезује је са VS Code окружењем.

Табела 4 приказује поделу одговорности. Ова подела је важна јер омогућава да се имплементација развија у слојевима. Парсер не мора да зна како се резултат приказује у едитору. LSP сервер не мора да зна детаље изградње CFG-а. Правила добијају припремљен контекст и враћају дијагностике.

Библиотечки улаз налази се у `src/lib.rs`. Из њега се извозе функције и типови које користе остали делови система: покретање CLI режима, покретање LSP сервера, parser API, Rule, RuleContext, default_rules и помоћне функције из `cfg` модула. То омогућава да спољни улази буду танки слојеви око истог језгра.

4.2 Главни ток анализе

Главни ток анализе почиње JavaScript изворним текстом, а завршава се листом дијагностика. Извор може доћи из датотеке, као у CLI режиму, или из отвореног документа у едитору, као у LSP режиму. После тога оба режима користе исту функцију `lint_file`.



Слика 3: Главни ток анализе у алату RustyJudge.

Слика 3 приказује централни ток. Прво се позива `parse_source`, која враћа синтаксички корен добијен помоћу `rslint_parser` библиотеке. Након успешног парсирања, RustyJudge покушава да изгради `SemanticModel`. Тај корак обухвата решавање симбола, изградњу графова и израчунавање података који су потребни за правила из каснијих фаза. Добијени подаци се заједно са изворним текстом и именом датотеке смештају у `RuleContext`. Затим `Linter` покреће сва правила и спаја њихове резултате у једну листу дијагностика.

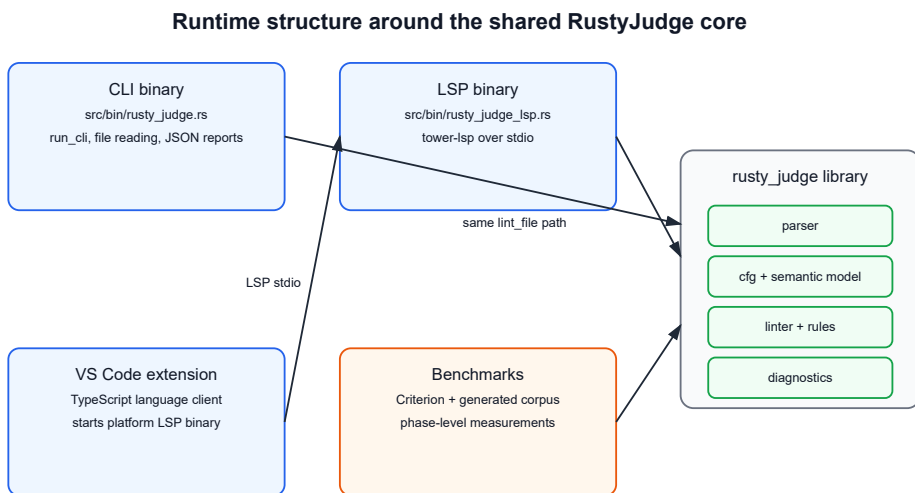
Посебан детаљ имплементације је да је семантички модел у контексту опционалан. У функцији `lint_file` изградња семантичког модела се покушава, али не прекида целу анализу ако не успе. У том случају правила која зависе само од текста или синтаксичке структуре могу и даље да се изврше, док правила која траже семантичке податке не пријављују резултат. Ово понашање представља облик постепене деградације: алат не мора да одбаци целу датотеку ако један дубљи слој анализе не може да се изгради.

Ова одлука има и ограничење. Ако семантичко спуштање не успе, правила која користе CFG, симболе или живост немају податке на основу којих би радила. Због тога је овај слој важан не само као имплементациона погодност, већ и као место на ком се одређује колико дубока анализа може да буде за конкретну датотеку.

4.3 Улазне тачке и заједничко језгро

RustyJudge има више начина покретања, али се они ослањају на исто језгро. CLI бинарни програм налази се у `src/bin/rusty_judge.rs`. Он само позива `run_cli()` и у случају грешке исписује поруку и враћа ненулти излазни код. LSP

бинарни програм налази се у `src/bin/rusty_judge_lsp.rs` и покреће асинхрони `run_lsp_server()`.



Слика 4: Улазне тачке које користе заједничко RustyJudge језгро.

Слика 4 приказује односе између режима рада. CLI чита датотеке са диска и резултат исписује као JSON. LSP сервер добија текст документа преко стандардног LSP протокола и резултат шаље назад едитору као дијагностике. Benchmark код заобилази кориснички приказ и позива исте библиотечке функције да би измерио појединачне фазе.

Овај распоред смањује дуплирање. Ако се измени правило или начин изградње контекста, промена важи и за CLI и за LSP. Ако се дода ново правило у `default_rules`, оно постаје део оба корисничка режима. Разлика између режима није у анализи, већ у начину на који се добија улаз и приказује излаз.

4.4 CLI режим

CLI режим је намењен покретању анализе из командне линије. Обрада аргумената налази се у `src/cli/`. Парсер аргумената је ручно имплементиран. Подржава директно навођење JavaScript датотека, једноставан облик директоријумског обрасца који се завршава са `/*` и опцију `--max-line-length`. Вредност за максималну дужину реда подразумевано је 120, али се може променити кроз аргумент командне линије.

Након обраде аргумената, `run_cli()` прави Linter са подразумеваним правилима, чита сваку изабрану датотеку и позива `lint_file`. За сваку датотеку формира се

FileReport, који садржи путању датотеке и листу дијагностика. На крају се сви извештаји серијализују помоћу `serde_json` и исписују на стандардни излаз.

Овај излазни формат је погодан за скрипте, benchmark помоћне алате или даљу обраду. Уместо да CLI директно форматира поруке као текст за човека, он враћа структуриране податке. Та одлука чини резултат предвидљивим и лакшим за аутоматску анализу.

4.5 Језгро linter-а и контекст правила

Језгро linter-а налази се у `src/linter/linter.rs`. Тип `Linter` садржи листу правила, представљену као `Vec<Box<dyn Rule + Send + Sync>>`. Свако правило имплементира исти интерфејс и добија исти `RuleContext`.

```
pub trait Rule: Send + Sync {  
    fn name(&self) -> &'static str;  
    fn check(&self, rule_context: &RuleContext) -> Vec<Diagnostic>;  
}
```

Листинг 4: Интерфејс који имплементира свако правило.

Листинг 4 приказује минимални уговор између језгра и правила. Правило има име и функцију `check`, која враћа листу дијагностика. Сам `Linter` не мора да зна шта правило проверава. Он само пролази кроз регистрована правила и додаје њихове резултате у заједничку листу.

`RuleContext` је централни објекат који повезује улазне податке и анализе. Садржи име датотеке, синтаксички корен, изворни текст, вредност за максималну дужину реда и опциони `SemanticModel`. Поред података, контекст садржи и помоћне методе за креирање дијагностика и приступ дубљим структурама. На тај начин правила не морају сама да претварају распоне у редове и колоне, нити да директно знају како је организована табела симбола.

Подразумевани скуп правила формира функција `default_rules`. У њему се налазе правила као што су `MaxLineLength`, `NoConsoleLog`, `NoDuplicateParams`, `NoEmptyBlock`, `StrictEquality`, `NoUnusedVars`, `NoVar`, `NoUnreachableCode`, `ConsistentReturn`, `NoDeadStores` и `NoInfiniteLoops`. Редослед у листи одређује редослед покретања, али правила не деле међусобно стање. Заједнички подаци долазе из `RuleContext` објекта.

4.6 Семантички модел као унутрашњи слој анализе

Модул `src/cfg/` представља унутрашњи слој који припрема податке за правила која не могу да раде само над непосредним изворним текстом. Главна функција овог модула је `build_semantic_model_from_root`. Она добија синтаксички корен и враћа `SemanticModel`.

`SemanticModel` садржи анализу скрипт нивоа, анализе функција и табелу симбола. Скрипт и свако тело функције имају сопствени `BodyAnalysis`, у коме се налазе CFG и резултат анализе живости. Функције су издвојене у посебне анализе, јер имају сопствене параметре, тело и контролни ток. Символи се чувају у заједничкој табели и идентификују се преко стабилних `SymbolId` вредности.

Изградња модела пролази кроз неколико имплементационих корака. Прво се прави `Resolver`, који бележи симболе и области важења. Затим се прави `CfgBuilder`, који постепено формира блокове и прелазе. `AstLowerer` обилази синтаксичко стабло и преводи релевантне конструкције у интерне структуре. Када се спуштање заврши, `builder` враћа CFG, над којим се рачуна живост. На крају се сви ови подаци пакују у `SemanticModel`.

У овом поглављу није потребно детаљно описивати алгоритме изградње CFG-а и живости, јер су они предмет наредних поглавља. За архитектуру је важније да је овај слој изолован. Правила не граде сама графове нити сама разрешавају симболе. Она само траже припремљене податке од `RuleContext` објекта.

4.7 Дијагностике и излазни формати

Заједнички формат дијагностике налази се у `src/diagnostics/`. Дијагностика садржи ред, колону и поруку. Тај формат је намерно једноставан, јер га користе различити излазни слојеви. CLI га серијализује у JSON, док LSP сервер исти резултат претвара у `tower-lsp` дијагностику.

Приликом креирања дијагностике правило најчешће не задаје ручно ред и колону. Уместо тога користи методе из `RuleContext`-а, као што су `diagnostic_at` и `diagnostic_at_span`. Оне користе распон из синтаксичког стабла или интерног `Span` типа и претварају га у координате погодне за приказ. Оваква подела помаже да правила остану усмерена на проверу, а не на формат излаза.

У LSP режиму свака `RustyJudge` дијагностика тренутно се приказује као упозорење. Распон у LSP дијагностици почиње на реду и колони из `RustyJudge` дијагностике и завршава се један карактер касније. Ово је једноставна мапа која је довољна за приказ проблема у едитору, али оставља простор за касније прецизније означавање целог израза или наредбе.

4.8 LSP сервер и VS Code проширење

LSP сервер је имплементиран у `src/lsp/lsp.rs` помоћу библиотеке `tower-lsp`. Сервер комуницира преко стандардног улаза и излаза, што омогућава да га покреће VS Code extension као засебан процес. Унутар сервера постоји `Backend` структура која чува LSP клијент, мапу отворених докумената, `Linter` са подразумеваним правилима и вредност за максималну дужину реда.

Сервер подржава отварање, измену, чување и затварање документа. При отварању и измени чува се цео текст документа. Затим се позива `lint_file`, а добијене дијагностике се претварају у LSP формат и објављују клијенту. При затварању документа сервер уклања текст из мапе и шаље празну листу дијагностика, чиме се проблеми уклањају из едитора.

VS Code проширење налази се у директоријуму `vscode-extension/`. Његов TypeScript код је танак клијент. При активацији проверава платформу и архитектуру, проналази одговарајућу Rust LSP бинарну датотеку у `server/` директоријуму и покреће је преко `stdio` транспорта. Проширење се активира за `javascript` и `javascriptreact` документе. На тај начин корисничко окружење остаје одвојено од саме анализе, која се извршава у Rust процесу.

4.9 Benchmark интеграција

Директоријум `benchmarks/` садржи генератор JavaScript корпуса, Criterion benchmark за RustyJudge и Node скрипте за покретање ESLint и RSLint мерења.

RustyJudge benchmark користи могућност да се библиотечке функције позову одвојено. Због тога се могу мерити различите фазе: само парсирање, парсирање са изградњом семантичког модела, извршавање правила над већ припремљеним контекстом и укупан linting процес. Ова подела је могућа зато што је архитектура система слојевита и зато што `lint_file` није једино место кроз које се интерни делови могу употребити.

Овај део архитектуре је важан за евалуацију, али се резултати мерења не разматрају у овом поглављу. Овде је битно само то да benchmark користи исти код који користе CLI и LSP. Тако се избегава ситуација у којој се мери понашање различито од стварног начина употребе алата.

4.10 Архитектонске последице

Оваква организација има неколико практичних последица. Прва је јасно одвајање улаза, анализе и излаза. CLI, LSP и benchmark имају различите потребе, али се све свде на исту анализу. Друга је могућност постепеног проширивања правила: ново правило може да користи само текст, само синтаксички корен или пуни семантички модел, у зависности од тога шта му је потребно. Трећа је могућност мерења појединачних фаза, што је значајно за касније поређење перформанси.

Постоје и ограничења. LSP сервер тренутно користи потпуну синхронизацију текста, па се при свакој измени анализира цела датотека. Све LSP дијагностике се приказују као упозорења. Конфигурација је ограничена, а вредност за максималну дужину реда у LSP режиму је фиксна. Семантички модел је опционалан, што омогућава наставак анализе, али може значити да дубља правила немају довољно

података. Ова ограничења не нарушавају основну архитектуру, али представљају природне правце за касније проширење.

У наредним поглављима ова архитектура се посматра детаљније кроз два најважнија унутрашња слоја: изградњу CFG-а и анализу живости. Тек након тога се приказује како појединачна правила користе припремљене податке.

Глава 5

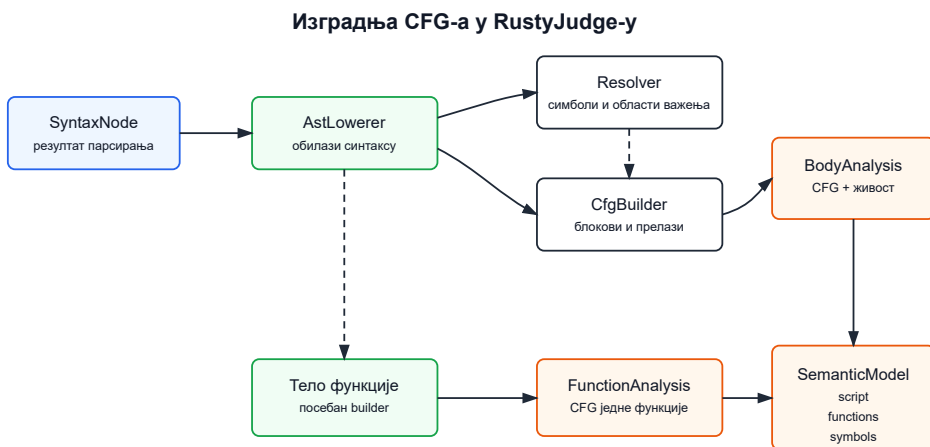
Имплементација контролно-проточног графа

Ово поглавље описује како је контролно-проточни граф реализован у алату RustyJudge. Посматра се модул `src/cfg/`, улога `CfgBuilder`-а и поступак спуштања JavaScript синтаксичког стабла у унутрашњи модел. Посебна пажња је посвећена начину на који се граде блокови, како се бележе дефиниције и употребе симбола и како се одвојено анализирају скрипт ниво и тела функција.

Имплементација CFG-а није изолована од остатка система. Она сарађује са разрешавањем симбола, правилима `linter`-а и анализом живости. Ипак, њен основни задатак је јасан: из синтаксичке структуре направити граф чији чворови садрже информације погодне за касније анализе.

5.1 Место CFG модула у пројекту

Сва логика везана за CFG налази се у директоријуму `src/cfg/`. Тај директоријум садржи више датотека, али се могу издвојити четири главне групе одговорности. Датотека `basic_blocks.rs` дефинише структуре података које представљају граф. Датотека `builder.rs` садржи имплементацију постепене изградње графа. Датотека `lowering.rs` повезује синтаксичко стабло са `builder`-ом. Датотека `resolver.rs` чува симболе и области важења које су потребне да би се дефиниције и употребе могле везати за стабилне идентификаторе.



Слика 5: Ток изградње CFG-a унутар src/cfg модула.

На слици 5 приказан је пут од синтаксичког корена до структура које користе правила. `AstLowerer` је место на ком се синтаксички чворови претварају у интерне кораке анализе. За имена и области важења ослања се на `Resolver`, док за саме блокове и прелазе позива `CfgBuilder`. Када се обрада једног тела заврши, резултат је `BodyAnalysis`: CFG тог тела и подаци израчунати над њим.

5.2 Основне структуре података

Главни тип за граф је `Cfg`. Он садржи идентификатор улазног блока, идентификатор излазног блока и вектор свих блокова. Блокови се не чувају као посебни објекти повезани показивачима, већ као елементи у вектору. Због тога је `BlockId` обичан индекс типа `usize`. Овакво решење поједностављује серијализацију, копирање и приступ блоковима током анализа.

```

pub struct Cfg {
    pub entry: BlockId,
    pub exit: BlockId,
    pub blocks: Vec<BasicBlock>,
}

pub struct BasicBlock {
    pub id: BlockId,
    pub statements: Vec<StatementInfo>,
    pub terminator: Terminator,
    pub predecessors: Vec<BlockId>,
    pub exception_successors: Vec<BlockId>,
}

```

Листинг 5: Скраћени приказ основних CFG структура.

Листинг 5 приказује најважнија поља. Сваки BasicBlock садржи редом додате исказе, терминатор, претходнике и посебну листу грана које воде ка catch блоку. Терминатор описује нормалан завршетак блока: безусловни прелаз, гранање, повратак из функције, недовршено стање или излазни блок. Посебна листа exception_successors омогућава да се поред редовног тока сачува и поједностављен модел преласка на обраду грешке.

Исказ унутар блока представљен је типом StatementInfo. Он садржи јединствени идентификатор исказа, врсту исказа, распон у изворном коду, симболе који се дефинишу и симболе који се користе. Ова структура је веза између CFG-а и касније анализе живости. Builder не памти само да се нека наредба налази у блоку, већ одмах бележи и које симболе та наредба чита или пише.

Врсте исказа су намерно сведене на оне које су потребне за анализе у пројекту: декларација променљиве, додела, ажурирање, израз, return, break, continue и празан исказ. То значи да CFG није потпуна реплика JavaScript AST-а. Он је средњи модел који задржава информације потребне за правила, а изоставља детаље који нису битни за тренутне анализе.

5.3 Улога CfgBuilder компоненте

CfgBuilder је mutable компонента која постепено гради граф. При креирању иницијализује два блока: улазни блок са идентификатором 0 и излазни блок са идентификатором 1. Током спуштања синтаксичког стабла builder памти тренутни блок, следећи идентификатор исказа, стек контекста петљи и тренутни циљ за изузетке.

Када се додаје обичан исказ, он се уписује у тренутни блок. Када се додаје конструкција која мења ток извршавања, builder поставља терминатор и по потреби креира нове блокове. На пример, return уписује исказ, поставља терминатор Return

и повезује тренутни блок са излазним блоком. После тога се креира нови блок који представља наставак у тексту програма, иако тај наставак може касније бити означен као недостижан.

Метод `finish` завршава изградњу графа. Ако је тренутни блок и даље отворен, повезује се са излазним блоком. Након тога се уклањају недостижни празни блокови, а идентификатори блокова се пресликавају тако да граф остане конзистентан. Овај корак није анализа правила, већ чишћење структуре која је настала током градње.

5.4 Спуштање синтаксичког стабла

Поступак спуштања почиње функцијом `build_semantic_model_from_root`. Она креира `Resolver`, `CfgBuilder` и вектор за анализе функција. Затим `AstLowerer` обилази корен синтаксичког стабла. Када се обилазак заврши, `script-level builder` се затвара, рачуна се живост за скрипт и формира се `SemanticModel`.

Пре обиласка наредби на скрипт нивоу, функцијске декларације се предекларишу. Ово је практичан начин да се приближи JavaScript понашању у ком су функцијске декларације доступне пре места на ком се текстуално појављују. Тек након тог корака спуштају се појединачне наредбе.

`AstLowerer` не прави исти третман за све синтаксичке чворове. Неке конструкције преводи у CFG исказе, неке само користи за прикупљање употреба симбола, а за функције покреће посебан поступак. Изрази који садрже угнежђене функције се обрађују пажљиво, да употребе симбола у унутрашњој функцији не би биле погрешно приписане спољашњем телу.

5.5 Декларације, доделе и употребе симбола

Када се обрађује декларација променљиве, `lowerer` издваја име декларатора и евентуалну иницијалну вредност. Име се региструје у `Resolver`-у као симбол одговарајуће врсте: `Var`, `Let` или `Const`. Ако постоји иницијализатор, из њега се прикупљају употребе симбола. Затим `builder` добија позив за додавање `VarDecl` исказа са листом дефинисаних и коришћених симбола.

Доделе се спуштају само када је лева страна једноставан идентификатор. То значи да је израз облика `x = ...` директно подржан, док сложенији облици као што су приступ пољу, деструктурирање или индексни приступ нису део тренутног модела. За сложене операторе доделе, као што је `+=`, симбол са леве стране се бележи и као употребљен и као дефинисан, јер нова вредност зависи од старе.

Ажурирања попут `x++` и `x--` представљају се као исказ који истовремено чита и пише исти симбол. Ово је важно за каснију анализу живости и правила која разликују корисну доделу од вредности која се никада не прочита.

5.6 Условне наредбе

За `if` наредбу `lowerer` издваја услов, затим грану која се извршава када је услов тачан и опциону `else` грану. Употребе симбола из услова се прослеђују `builder`-у. `Builder` креира блок за `then` грану, блок за `else` грану и, ако је потребно, заједнички блок наставка.

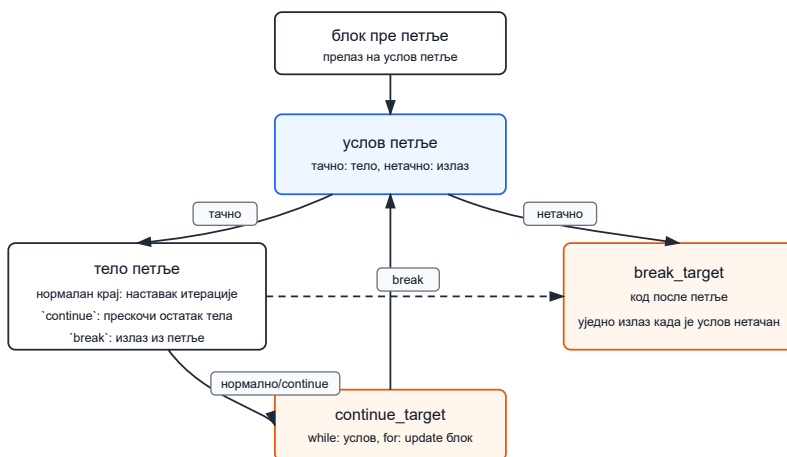
Ако обе гране прекидају ток, на пример обе садрже `return`, не постоји отворена грана која треба да пређе у наставка. У том случају `builder` креира нови блок наставка који може бити недостижан. Овај дизајн је користан јер каснија правила могу да пријаве код који се налази након такве конструкције.

Услов се у CFG-у не чува као цело синтаксичко стабло израза. Чува се скуп симбола који се у услови користе. За тренутна правила то је довољно, јер је битно да анализа зна које вредности услов чита, а не да поново тумачи комплетан израз.

5.7 Петље и циљеви за `break` и `continue`

Петље захтевају додатно стање током изградње графа. `CfgBuilder` зато користи стек `LoopContext` вредности. Сваки контекст памти `break_target` и `continue_target`. Када се у телу петље појави `break`, `builder` га повезује са циљем за излаз из петље. Када се појави `continue`, повезује га са циљем за наставка петље, што је код `while` петље условни блок, а код `for` петље може бити `update` блок.

Циљеви `'break'` и `'continue'` наредби у петљи



Слика 6: Циљеви које `CfgBuilder` користи за `break` и `continue` у петљи.

Слика 6 приказује само циљеве које `builder` памти док спушта тело петље. Када тело заврши нормално, ток иде ка `continue_target`. Иста мета се користи и за `continue`, с тим што се тада прескачу преостале наредбе у телу петље. Наредба `break` води

директно у `break_target`, односно у блок који представља наставак програма после петље.

`while` петље додатно користе једноставну процену константне истинитости услова. Ако је услов јасно увек тачан, `builder` може да направи безусловни прелаз у тело петље. Ако је услов јасно нетачан, може да пређе директно у излазни блок петље. За све остале услове чувају се употребе симбола и гради се уобичајено гранање. Ова процена не покушава да буде потпуна JavaScript евалуација, већ практична помоћ за једноставне случајеве као што су `while (true)` или `while (0)`.

`for` петља се спушта у више корака. Иницијализатор се обрађује пре уласка у структуру петље. Услов се смешта у посебан блок. Тело петље се спушта као посебан део, а `update` израз добија свој блок када постоји. Ова подела омогућава да `continue` у `for` петљи води ка `update` делу, а не директно на проверу услова.

`for-in` и `for-of` петље деле заједничку функцију за спуштање. Лева страна може бити декларација променљиве или постојећи идентификатор. Десна страна се третира као израз чије се употребе бележе у заглављу петље. Циљна променљива се моделује као додела која се дешава при итерацији.

5.8 Функције као посебна тела анализе

Функције у RustyJudge-у не деле CFG са местом на ком су декларисане. Свака функција добија сопствени `builder`, сопствени CFG и сопствени резултат анализе живости. Ово важи за функцијске декларације, функцијске изразе и `arrow` функције.

При спуштању функције `lowerer` улази у нову област важења. Параметри се декларишу као `SymbolKind::Param`. Ако је у питању именовани функцијски израз, унутрашње име функције се такође може декларисати у одговарајућем опсегу. Након тога се спушта тело функције. Код `arrow` функција са изразом као телом, израз се моделује као имплицитни `return`.

Резултат овог поступка је `FunctionAnalysis`, који садржи име функције ако постоји, симбол функције, распон, параметре и `BodyAnalysis`. Сви `FunctionAnalysis` објекти чувају се у `SemanticModel.functions`. Скрипт ниво се чува одвојено у `SemanticModel.script`.

Ова подела спречава да ток извршавања унутар функције буде помешан са током извршавања у спољашњем коду. Декларација функције на спољашњем нивоу утиче на табелу симбола, али тело функције има свој контролни ток.

5.9 try/catch модел

RustyJudge садржи поједностављену подршку за try/catch. Lowerer издваја try блок и catch блок из синтаксе. Ако постоји параметар catch клаузе, он се уводи као симбол у посебној области важења за catch блок.

Builder током изградње try дела памти тренутни catch циљ. Када се у том делу дода исказ или гранање, builder може да дода и грану ка catch блоку. Такве гране се чувају у пољу exception_successors, одвојено од терминатора. При израчунавању наследника блока, редовни прелази и прелази ка catch блоку спајају се у једну листу.

Овај модел је намерно једноставан. Он омогућава да CFG има везу од кода унутар try блока ка catch блоку, али не моделира све детаље JavaScript изузетака. Тренутна имплементација не садржи посебан терминатор за throw, не моделира finally блок и не разликује прецизно које наредбе стварно могу бацити изузетак. Због тога се ова подршка може описати као приближан модел тока који прелази на обраду грешке, а не као потпуна имплементација JavaScript exception семантике.

5.10 Завршавање и чишћење графа

Током изградње CFG-а могу настати празни блокови. Неки су потребни као циљеви гранања или места наставка, али неки могу остати недостижни и без исказа. Метод finish зато након затварања графа позива поступак који уклања недостижне празне блокове.

Уклањање блокова захтева пажљиво пресликавање идентификатора. Ако се блокови избаце из вектора, сви терминатори, претходници и гране ка catch блоковима морају да показују на нове идентификаторе. Због тога се прво прави мапа старих идентификатора у нове, а затим се реконструише листа блокова и сви прелази.

Ово чишћење не уклања блокове који садрже исказе, чак и ако су недостижни. Такви блокови су важни за правило које пријављује недостижан код. Уклањају се само празни недостижни блокови који су настали као технички производ конструкције графа.

5.11 Тренутни обим подршке

Тренутни CFG слој подржава конструкције које су потребне за правила имплементације у RustyJudge-у. То укључује декларације променљивих, изразе, једноставне доделе, ажурирања, return, блокове, if, while, for, for-in, for-of, break, continue, функције и поједностављени try/catch.

Постоје и позната ограничења. Доделе су ограничене на једноставне идентификаторе са леве стране. Деструктурирање, приступ својству и индексни приступ нису

моделовани као дефиниције симбола. `switch`, `do-while`, `throw`, `finally` и прецизно кратко спајање логичких оператора нису део тренутног CFG модела.

Ова ограничења су важна за тумачење резултата рада. Циљ имплементације није био да покрије целу ECMAScript семантику, већ да изгради довољно стабилан CFG слој за правила која се анализирају у наставку рада. Наредно поглавље користи овај CFG као основу за анализу живости података.

Глава 6

Имплементација анализе живости података

У RustyJudge-у анализа живости представља фазу семантичког модела која над CFG-ом рачуна које тренутне вредности симбола могу бити прочитане у наставку извршавања. Улаз у ову фазу је Cfg са основним блоковима, терминаторима и StatementInfo записима у којима су већ попуњене листе uses и defines. Излаз је LivenessResult, структура која садржи достижне блокове, блоковске use/def скупове, live_in/live_out скупове и живост на нивоу појединачних исказа.

Фаза живости не приступа поново синтаксичком стаблу и не разрешава имена. Ослања се на податке које је CFG слој већ припремио. На тај начин је граница између фаза јасна: AstLowerer и Resolver одређују који се симболи читају и пишу, док анализа живости израчунава да ли су њихове тренутне вредности потребне у наставку извршавања.

6.1 Место анализе у семантичком моделу

Главна функција анализе налази се у датотеци src/cfg/liveness.rs и зове се compute_liveness. Позива се након што CfgBuilder заврши граф. То се дешава посебно за скрипт ниво и посебно за свако тело функције. Због тога сваки BodyAnalysis у семантичком моделу садржи два блиско повезана податка: cfg и liveness.

Живост се рачуна унутар једног тела анализе. Позив функције не спаја CFG позиваоца са CFG-ом позване функције. RustyJudge зато анализира скрипт и сваку функцију као засебне јединице, а правила касније пролазе кроз све добијене анализе. Модел је интрапроцедуралан: прецизан је у границама једног тела, без изградње позивног графа целог програма.



Слика 7: Кораци које RustyJudge извршава при рачунању живости.

Слика 7 приказује редослед корака. Из CFG-а се најпре издвајају достижни блокови. Затим се за сваки блок формирају локални скупови use и def. После тога се итеративно рачунају live_in и live_out скупови на нивоу блокова. На основу блоковских резултата рачуна се живост на нивоу исказа, која је неопходна за проверу да ли је конкретна додела прочитана пре следећег уписа у исти симбол.

6.2 Структура LivenessResult

Резултат анализе је структура LivenessResult. Она не чува имена променљивих, већ SymbolId вредности. Име се добија преко табеле симбола у SemanticModel-у тек када је потребно формирати поруку за корисника. Такво представљање спречава мешање симбола који имају исто име, али припадају различитим областима важења.

```

pub struct LivenessResult {
    pub reachable: HashSet<BlockId>,
    pub block_use: Vec<HashSet<SymbolId>>,
    pub block_def: Vec<HashSet<SymbolId>>,
    pub live_in: Vec<HashSet<SymbolId>>,
    pub live_out: Vec<HashSet<SymbolId>>,
    pub stmt_live_in: Vec<Vec<HashSet<SymbolId>>>,
    pub stmt_live_out: Vec<Vec<HashSet<SymbolId>>>,
}
  
```

Листинг 6: Скраћени приказ структуре LivenessResult.

Листинг 6 показује да су резултати организовани као вектори. Пошто је BlockId у RustyJudge-у индекс у cfg.blocks, исти идентификатор се користи и као индекс у block_use, block_def, live_in и live_out векторима. За живост исказа потребан је још један ниво индексирања: прво се бира блок, а затим позиција исказа у том блоку.

Табела 5: Подаци које RustyJudge чува после анализе живости.

Поље	Значење у имплементацији
reachable	Блокови до којих постоји пут од entry блока. Правила га користе да раздвоје код који може да се изврши од кода који је остао иза прекида тока.
block_use	Симболи који се у блоку читају пре него што их тај блок дефинише.
block_def	Симболи којима блок додељује нову вредност или их уводи као дефиницију.
live_in	Симболи чија тренутна вредност може бити потребна при уласку у блок.
live_out	Симболи чија тренутна вредност може бити потребна после изласка из блока.
stmt_live_in	Симболи живи непосредно пре појединачног исказа.
stmt_live_out	Симболи живи непосредно после појединачног исказа.

Резултат има две грануларности. Блоковски скупови преносе информацију кроз граф, док скупови на нивоу исказа омогућавају прецизније провере унутар блока. Правило no-dead-stores не може да се ослони само на стање после целог блока, јер мора да зна да ли је вредност додељена у тачно одређеном исказу жива непосредно после тог исказа.

6.3 Достижност као први корак

Пре саме живости RustyJudge израчунава скуп достижних блокова. Функција compute_reachable_blocks креће од cfg.entry блока и обилази наследнике. Наследници се добијају преко метода BasicBlock::successors. Тај метод спаја редовне прелазе из терминатора и гране које воде ка catch блоку, ако постоје.

Скуп reachable има две улоге. Итеративна анализа живости не обрађује блокове који не могу да се изврше, а правила касније користе исти резултат. На пример, no-unreachable-code пријављује исказе у блоковима који нису у овом скупу, док no-unused-vars игнорише употребе симбола које се налазе само у недостижном делу кода.

Достижност зато остаје део LivenessResult структуре. Иако се рачуна пре живости, њена вредност није само помоћна; она је посебан резултат семантичког слоја који користе правила.

6.4 Формирање use и def скупова

Након достижности, RustyJudge формира локалне скупове за сваки блок. Функција `compute_block_use_def` пролази кроз исказе у редоследу у ком се налазе у блоку. За сваки исказ чита симболе из `uses` и `defines` листа. Ако је симбол употребљен пре него што је дефинисан у том истом блоку, улази у `use` скуп тог блока. Сваки симбол који исказ дефинише улази у `def` скуп.

У овом контексту дефиниција означава упис нове вредности у симбол, а не само прву декларацију променљиве у програму. Додела `x = 5` дефинише нову вредност симбола `x`. Изрази `x++` и сложене доделе читају стару вредност и уписују нову, па се исти симбол појављује и у `uses` и у `defines`.

Употребе из терминатора блока додају се у исти локални резултат. Ако је терминатор гранање, у `use` улазе симболи из услова. Ако је терминатор `return`, у `use` улазе симболи из повратне вредности. Услов и `return` нису увек обични искази у листи `statements`, али и даље читају вредности које морају бити обухваћене анализом.

6.5 Израчунавање блоковске живости

Када су `block_use` и `block_def` попуњени, `compute_liveness` иницијализује празне `live_in` и `live_out` скупове за све блокове. Затим покреће петљу која се извршава док год се у некој итерацији промени бар један скуп. Унутар петље блокови се пролазе обрнутим редоследом, што одговара смеру анализе живости: информација се преноси од каснијих тачака програма ка ранијим.

За сваки достижан блок најпре се прави нови `live_out`. Он настаје као унија `live_in` скупова свих достижних наследника. Пошто наследници долазе из метода `successors`, исти код природно обухвата обичне прелазе, гранања, повратак ка излазном блоку и поједностављене прелазе ка `catch` блоку.

Нови `live_in` почиње као копија `block_use` скупа. Затим се додају симболи из новог `live_out` скупа, али само ако нису у `block_def` скупу тог блока. Семантички, вредност која је потребна после блока мора бити жива и пре блока, осим ако блок пре те употребе уписује нову вредност истог симбола. Ако се нови скупови разликују од старих, поставља се `changed = true` и анализа наставља још једну итерацију.

Имплементација користи директне операције над `HashSet<SymbolId>` скуповима: копирање, унију, проверу припадности и убацивање. У коду не постоји посебна апстракција за општи `data-flow framework`, јер пројекат тренутно има једну конкретну анализу тог типа. Такво решење смањује количину инфраструктурног кода и оставља алгоритам видљивим у самој функцији `compute_liveness`.

6.6 Живост на нивоу исказа

Блоковска живост описује стање на границама блокова. За проверу појединачних додела потребна је финија информација: скуп живих симбола непосредно пре и после сваког исказа. Због тога RustyJudge после фиксне тачке позива `compute_statement_liveness`.

Ова функција обрађује сваки блок од краја ка почетку. Почетни скуп за блок је његов `live_out`. У тај скуп се додају и употребе из терминатора блока, јер услов гранања или `return` читају вредности после последњег обичног исказа у блоку. Затим се искази пролазе у обрнутом редоследу.

За сваки исказ се прво запамти тренутни скуп као `stmt_live_out`, односно стање непосредно после исказа. Потом се из копије тог скупа уклањају симболи које исказ дефинише, јер претходна вредност није жива пре уписа нове вредности. На крају се додају симболи које исказ користи, јер они морају бити доступни пре извршавања исказа. Добијени скуп постаје `stmt_live_in`.

Овај резултат повезује блоковску анализу са правилима која раде над конкретним исказима. Блок може да има исправан `live_out`, али правило за мртве доделе захтева податак да ли је вредност уписана у једном исказу жива одмах после тог исказа.

6.7 Пример мртве доделе

У примеру из листинга 7 променљива `result` се користи у `return` наредби, али вредност уписана у првом исказу не стиже до тог читања.

```
function choose(a, b) {
  let result = a;
  result = b;
  return result;
}
```

Листинг 7: Пример у ком је прва вредност променљиве `result` мртва.

Симбол `result` у овом примеру није некоришћена променљива, јер се чита у `return result`. Мртва је само почетна вредност добијена из `a`: она се уписује у `result`, а затим се пре првог читања замењује вредношћу `b`. Живост на нивоу исказа раздваја употребу променљиве од употребе конкретне вредности уписане у њу.

Табела 6: Живост на нивоу исказа за пример из листинга 7.

Исказ	uses	defines	live-out	Значење
let result = a	{a}	{result}	{b}	Вредност уписана у result није жива после исказа.
result = b	{b}	{result}	{result}	Нова вредност result је жива јер ће бити враћена.
return result	{result}	{}	{}	result се чита у повратној вредности.

Табела 6 приказује суштину правила no-dead-stores. После првог исказа у live-out скупу нема симбола result. То значи да вредност која је управо уписана у result неће бити прочитана пре следећег уписа у исти симбол. После другог исказа result јесте жив, јер return чита његову нову вредност.

6.8 Ограничења тренутног модела

Тренутна имплементација ради на нивоу симбола. Она не прати поља објеката, елементе низова нити алијасе. Додела `obj.x = 1` не постаје дефиниција посебног симбола `obj.x`, јер такав симбол у моделу не постоји. Слично томе, анализа не утврђује да ли две променљиве показују на исти објекат.

Друго ограничење је интрапроцедуралност. Живост се рачуна унутар једног тела: скрипта или функције. Анализа не прати како вредност пролази кроз позиве функција, нити закључује да ли ће нека функција сигурно прочитати аргумент. За linter правила у овом пројекту то је прихватљиво ограничење, јер омогућава стабилне локалне дијагностике без сложене међупроцедуралне анализе.

Тачност живости зависи од тачности података које је CFG слој уписао у uses и defines. Ако нека JavaScript конструкција није моделована као дефиниција или употреба симбола, анализа живости је неће сама открити. Проширивање подршке за нове синтаксичке облике зато почиње у CFG слоју: када StatementInfo садржи исправне дефиниције и употребе, анализа живости може да их користи без промене основног алгорита.

Оваква организација омогућава да правила не реконструишу ток података самостално. Она добијају припремљен резултат семантичког модела и користе га само за сопствену проверу.

Глава 7

Имплементација правила

Скуп подразумеваних правила у RustyJudge-у покрива три нивоа провере: директну проверу текста, проверу синтаксичког стабла и проверу података добијених из семантичког слоја. Разлика између ових група одређује које податке правило тражи од `RuleContext` објекта и колико је дубока анализа потребна да би дијагностика била поуздана.

Сва правила враћају исту врсту резултата: листу `Diagnostic` објеката. Разлике су у начину на који долазе до места проблема. Једноставна правила користе позицију токена или реда у изворном тексту. Правила која зависе од CFG-а користе `Span` вредности сачуване у `StatementInfo` и `FunctionAnalysis` структурама.

7.1 Преглед имплементираних правила

Подразумевани скуп правила формира се у функцији `default_rules`. Сва правила су независна: не мењају заједничко стање и не позивају једно друго. Заједнички су им само улазни контекст и формат дијагностике.

Табела 7: Правила и подаци који су им потребни.

Правило	Потребни подаци	Услов за дијагностику
max-line-length	Изворни текст и max_line_length.	Ред има више карактера од дозвољене границе.
no-console-log	Токени из синтаксичког стабла.	Појављује се низ токена console, ., log.
no-duplicate-params	PARAMETER_LIST чворови и идентификатори у њима.	Исто име параметра појављује се више пута у истој листи параметара.
no-empty-block	BLOCK_STMT чворови и њихови токени.	После { следи } без садржаја, уз игнорисање whitespace токена.
strict-equality	Токени оператора.	Користи се == или != уместо === или !==.
no-var	VAR_DECL чворови.	Декларација користи кључну реч var.
no-unused-vars	Табела симбола, употребе симбола и достижност.	Симбол проверљиве врсте нема употребу у достижном коду.
no-unreachable-code	CFG и скуп достижних блокова.	Исказ се налази у блоку који није достижан од улаза у тело.
consistent-return	CFG функције, терминатори и достижни претходници излазног блока.	Неке достижне путање враћају вредност, а друге не враћају вредност.
no-dead-stores	Живост на нивоу исказа.	Исказ уписује вредност у симбол који није жив после тог исказа.
no-infinite-loops	CFG, достижност, SCC компоненте и пут до излазног блока.	Циклична компонента нема достижан излаз из тела анализе.

7.2 Правила над текстом и синтаксом

Правило max-line-length ради директно над изворним текстом. Обилази редове из RuleContext.source и пореди дужину реда са вредношћу RuleContext.max_line_length. Дијагностика се поставља у колону која прва прелази дозвољену границу. Ово правило је независно од JavaScript синтаксе: једнако ради над сваким редом који стигне до linter-a.

Правило `strict-equality` користи синтаксичке токене. Обилази елементе стабла и проверава да ли је врста елемента `EQ2` или `NEQ`. Прво одговара оператору `==`, а друго оператору `!=`. У оба случаја правило пријављује позицију самог оператора. Не анализира типове израза, јер циљ правила није да процени да ли је нестрого поређење у конкретном случају безбедно, већ да наметне доследан стил поређења.

Правило `no-var` тражи `VAR_DECL` чворове и у њима кључну реч `var`. Провера је синтаксичка, јер је употреба `var` видљива већ у `AST`-у. За дијагностику није потребно знати у којој области важења је променљива декларисана, нити да ли се касније користи.

Правило `no-console-log` обилази токене и тражи тачан низ `console`, `.`, `log`. Због такве имплементације оно пријављује и израз `console.log` без заграда, не само позив `console.log(...)`.

Правило `no-empty-block` ради над `BLOCK_STMT` чворовима. После леве витичасте заграде прескаче `whitespace` токене и проверава да ли је следећи токен десна витичаста заграда.

Правило `no-duplicate-params` обилази сваку листу параметара посебно. За једну `PARAMETER_LIST` структуру прави локални `HashSet<String>` и у њега додаје имена параметара. Ако се име већ налази у скупу, пријављује се дијагностика на поновљеном идентификатору. Скуп се прави поново за сваку листу параметара, па исто име у две различите функције није проблем.

7.3 Правила над симболима и достижношћу

Правило `no-unused-vars` користи табелу симбола и податке о употребама симбола. Прво сакупља све симболе који се читају у достижним блоковима. У обзир улазе `uses` листе исказа, употребе из услова гранања и употребе из `return` терминатора. Након тога правило пролази кроз симболе из табеле и пријављује оне који припадају проверљивим врстама: `Var`, `Let`, `Const`, `Param` и `Function`.

Достижност је важна за ово правило зато што употреба у коду који се не може извршити не треба да сакрије проблем. На пример, ако се променљива чита само после `return` наредбе, та употреба се не рачуна. Правило ради над `SymbolId` вредностима, па сенчење имена не прави лажне резултате: две променљиве са истим текстуалним именом остају различити симболи.

Правило `no-unreachable-code` користи исти скуп достижних блокова, али на директнији начин. Пролази кроз блокове сваког тела анализе и пријављује сваки исказ у блоку који није достижан. Порука се везује за `Span` исказа. Ако је недостижан код у функцији, порука садржи и име функције када је оно доступно.

7.4 Правила над повратним путањима

Правило `consistent-return` проверава само функције. За сваку функцију посматра претходнике излазног блока који су достижни. Такви претходници описују начине на које извршавање може да напусти функцију: преко `return` терминатора, преко обичног пада на крај функције или преко гранања које води директно ка излазу.

Правило бележи две појаве. Прва је постојање достижне путање која завршава `return` наредбом са вредношћу. Друга је постојање достижне путање која завршава `return` наредбом без вредности или доласком до краја функције без експлицитног повратка вредности. Дијагностика се пријављује само када постоје обе појаве у истој функцији.

Таква провера не захтева анализу типова. Правило не испитује шта се враћа, већ да ли све релевантне путање имају исти облик повратка. Функција која на свим путањима враћа вредност не пријављује се. Функција која нигде не враћа вредност такође се не пријављује. Проблем је мешање ова два стила унутар исте функције.

7.5 Правило за мртве доделе

Правило `no-dead-stores` је најдиректнији корисник живости на нивоу исказа. Проверава само исказе који уписују вредност: декларације са иницијализатором, доделе и `update` изразе. То је издвојено у функцији `is_store_statement`.

```
fn is_store_statement(kind: &StatementKind) -> bool {
    matches!(
        kind,
        StatementKind::Assign
        | StatementKind::Update
        | StatementKind::VarDecl { has_initializer: true }
    )
}
```

Листинг 8: Врсте исказа које правило `no-dead-stores` посматра као упис вредности.

За сваки такав исказ правило чита `stmt_live_out` скуп. Ако симбол који је исказ дефинисао није жив после исказа, додељена вредност не може бити прочитана пре следећег уписа или краја тела. Тада се пријављује дијагностика на распон исказа. Недостижни блокови се прескачу, јер се за њих већ може пријавити проблем недостижног кода.

```
let x = 1;
x = 2;
console.log(x);
```

Листинг 9: Пример у ком је прва додела променљивој `x` мртва.

У листингу 9 прва вредност променљиве `x` не стиже до читања. Друга додела је корисна, јер се управо та вредност чита у `console.log(x)`. Правило зато не посматра само да ли се променљива негде користи, већ да ли се користи конкретна вредност настала датим уписом.

Ово правило зависи од тачности `defines` података. Тренутно су подржани уписи у једноставне идентификаторе и декларације које CFG слој уме да представи као `StatementInfo`. Сложени облици уписа, као што су дејструктурирање или својства објеката, не могу бити прецизно пријављени док нису моделовани у CFG слоју.

7.6 Правило за бесконачне петље

Правило `no-infinite-loops` користи структуру CFG-а као граф. За свако тело анализе прво рачуна блокове из којих се може стићи до излазног блока. То ради обрнутим обиласком преко `predecessors` листа, почевши од `cfg.exit`. Затим израчунава јако повезане компоненте над достижним делом графа.

Компонента се сматра цикличном ако садржи више од једног блока или ако једини блок у компоненти има грану ка самом себи. Ако ниједан блок из такве компоненте не може да стигне до излазног блока, правило пријављује потенцијалну бесконачну петљу.

Ова провера не покушава да докаже све могуће облике бесконачног извршавања. Она препознаје ситуације у којима CFG нема излазну путању из циклуса. Због тога петља са достижним `break` прелазом неће бити пријављена, чак и ако би услови у реалном извршавању могли спречити да се тај `break` икада достигне. То је свесно ограничење: правило се ослања на структуру путања, а не на вредносну анализу услова.

7.7 Дијагностике и тестови

Правила не граде излазни формат директно. Она користе `diagnostic_at` када имају `TextRange` из синтаксичког стабла и `diagnostic_at_span` када имају интерни `Span` из CFG слоја. Оба метода на крају производе исти `Diagnostic`, са редом, колоном и поруком. Због тога CLI и LSP слој могу да користе резултате правила без познавања начина на који је правило дошло до позиције.

Свака имплементација правила садржи јединичне тестове. Тестови покривају основне позитивне случајеве, случајеве у којима не треба пријавити проблем и граничне случајеве. Код правила која зависе од CFG-а тестови су посебно важни, јер проверавају понашање у гранањима, петљама, функцијама и недостижном коду. На тај начин тестови служе као спецификација очекиваног понашања правила, не само као провера да код тренутно пролази.

Глава 8

CLI и VS Code интеграција

RustyJudge има два корисничка улаза: покретање из командне линије и рад у едитору преко VS Code проширења. Оба режима користе исто језгро анализе. Разлика је у начину на који добијају изворни текст и у облику у ком приказују дијагностике.

8.1 CLI режим

CLI бинарна датотека налази се у `src/bin/rusty_judge.rs`. Њена улога је мала: позива `run_cli`, исписује грешку ако до ње дође и враћа ненулти излазни код. Сама обрада аргумената налази се у `src/cli/`, док се извршавање анализе налази у `src/run_cli/run.rs`.

CLI прихвата путање до `.js` датотека и једноставан облик директоријумског обраца који се завршава са `/*`. У том случају се из директоријума узимају само JavaScript датотеке. Подржана је и опција `--max-line-length`, која мења границу за правило `max-line-length`. Подразумевана вредност је 120.

После обраде аргумената прави се Linter са подразумеваним правилима. За сваку датотеку чита се изворни текст, позива се `lint_file`, а резултат се смешта у `FileReport`. На крају се цела листа извештаја исписује као JSON. Такав излаз је погодан за `benchmark` скрипте и аутоматску обраду, јер није везан за конзолни приказ намењен само човеку.

```
[
  {
    "file": "example.js",
    "diagnostics": [
      { "row": 3, "col": 5, "message": "..." }
    ]
  }
]
```

Листинг 10: Облик CLI резултата.

CLI режим је користан и као најједноставнији начин провере рада алата. Не зависи од едитора, LSP комуникације или VS Code паковања, па је погодан за тестирање језгра анализе над конкретним датотекама.

8.2 LSP сервер

За рад у едитору RustyJudge користи Language Server Protocol. LSP раздваја едитор од алата који анализира код. Едитор шаље догађаје о документима, а сервер враћа одговоре у стандардном облику, најчешће као дијагностике, аутодопуне, дефиниције или друге језичке функционалности. У RustyJudge-у је имплементиран само део потребан за дијагностике.

LSP бинарна датотека налази се у `src/bin/rusty_judge_lsp.rs`. Покреће `run_lsp_server` у `tokio runtime`-у. Сервер је реализован помоћу библиотеке `tower-lsp`, која обезбеђује структуру за JSON-RPC комуникацију и `Rust trait LanguageServer`. Комуникација иде преко стандардног улаза и излаза, што је уобичајен начин покретања `language server`-а из едитора.

Главна структура LSP дела је `Backend`. Она чува `Client`, мапу отворених докумената, један `Linter` и вредност за максималну дужину реда. Документи се чувају у `RwLock<HashMap<Url, String>>`, јер LSP методе раде асинхроно и више догађаја може приступати истом стању.

При иницијализацији сервер објављује да подржава отварање и затварање докумената, чување и FULL синхронизацију измена. То значи да при промени документа сервер добија цео нови текст, а не само разлику између две верзије. Ово је једноставније за имплементацију и довољно је за величину и циљ пројекта.

Када се документ отвори или измени, сервер ажурира текст у мапи и позива `lint_and_publish`. Та функција покреће `lint_file` над тренутним текстом, претвара RustyJudge дијагностике у LSP дијагностике и шаље их едитору преко `publish_diagnostics`. Ако парсирање или анализа врати грешку, сервер шаље једну дијагностику на почетак датотеке са поруком о синтаксичкој грешци.

Конверзија дијагностике је директна. RustyJudge користи ред и колону који су индексирани од један, док LSP користи позиције индексирани од нуле. Зато се при конверзији обе вредности умањују за један. Све дијагностике се шаљу као `WARNING`, са извором `rusty-judge`. Распон тренутно покрива један карактер од позиције дијагностике.

Када се документ затвори, сервер га уклања из мапе и шаље празну листу дијагностика. То чисти приказ проблема у едитору за документ који више није отворен.

8.3 VS Code проширење

VS Code проширење се налази у директоријуму `vscode-extension/`. Клијент је написан у TypeScript-у, у датотеци `src/extension.ts`. Његов задатак није да анализира JavaScript, већ да пронађе RustyJudge LSP бинарну датотеку, покрене је и повеже је са VS Code LSP клијентом.

Проширење се активира за језике `javascript` и `javascriptreact`. При активацији одређује платформу и архитектуру процеса, на пример `win32-x64`, и на основу тога гради очекивану путању до `server` бинарне датотеке. На Windows-у очекује `rusty_judge_lsp.exe`, док на осталим системима очекује `rusty_judge_lsp`. Ако платформа није подржана или бинарна датотека не постоји, кориснику се приказује грешка.

Када је бинарна датотека пронађена, проширење прави `ServerOptions` са `TransportKind.stdio`. Затим прави `LanguageClient` са `selector`-ом за JavaScript документе и покреће клијент. Од тог тренутка VS Code шаље LSP догађаје серверу, а сервер враћа дијагностике које се приказују у едитору и у панелу Problems.

Паковање проширења ослања се на `package.json`, `esbuild` и `vsce`. TypeScript код се компајлира и пакује у `dist/extension.js`, а LSP бинарна датотека мора бити присутна у одговарајућем `server/<platform>/` директоријуму пре паковања VSIX архиве. На тај начин VS Code проширење остаје танак клијент, док се сва анализа и даље извршава у Rust бинарном програму.

Ова организација задржава један извор истине за правила. CLI, LSP сервер и VS Code проширење не садрже различите имплементације провера. Они се разликују само по начину покретања и приказа резултата.

Глава 9

Мерење перформанси

Перформансе RustyJudge-a мерене су над генерисаним JavaScript корпусом у три величине: `small`, `medium` и `large`. Корпус је направљен скриптом `generate_benchmark_corpus.py`, која понавља исти шаблон функције са условом, петљама, позивом функције и `console.log` исказом. Добијене датотеке имају 125, 1250 и 7500 линија.

RustyJudge је мерен помоћу Criterion benchmark-a. Одвојено су мерени парсирање, изградња семантичког модела и CFG-a, извршавање правила и укупан ток анализе. ESLint је мерен преко Node API-ја и његових stats података. RSLint је мерен преко CLI позива, па је за њега доступно само укупно wall-clock време.

Табела 8: Резултати мерења над генерисаним корпусом.

Корпус	Алат	Parse time	Изградња моде- ла и CFG-a	Rule time	Total time
small	RustyJudge	0.29 ms	0.57 ms	0.41 ms	0.95 ms
small	ESLint	2.42 ms	N/A	0.40 ms	5.90 ms
small	RSLint	N/A	N/A	N/A	22.59 ms
medium	RustyJudge	4.60 ms	7.30 ms	4.49 ms	11.52 ms
medium	ESLint	16.20 ms	N/A	1.94 ms	28.73 ms
medium	RSLint	N/A	N/A	N/A	23.11 ms
large	RustyJudge	161.25 ms	177.49 ms	42.97 ms	222.96 ms
large	ESLint	95.39 ms	N/A	9.92 ms	146.40 ms
large	RSLint	N/A	N/A	N/A	23.16 ms

Табела 8 приказује просечна времена у милисекундама. Код RustyJudge-a се види да највећи део времена код већих датотека одлази на парсирање и изградњу модела са CFG-ом, док је само извршавање правила мањи део укупног времена. На малом и средњем корпусу RustyJudge има мање укупно време од ESLint-a у овом мерењу. На великом корпусу ESLint има мање укупно време, док RustyJudge плаћа већу цену изградње унутрашњих структура.

RSLint резултат није фазно упоредив са остала два алата, јер benchmark бележи само укупно време CLI процеса. Зато су остала поља означена као N/A. Исто важи и за

колону изградње модела и CFG-а код ESLint-а, јер тај алат не излаже упоредиву фазу у коришћеном benchmark интерфејсу.

Ове резултате треба посматрати као индикативно поређење. Скуп правила није потпуно идентичан у сва три алата, а начин мерења није исти за сваки алат. Ипак, мерење показује где RustyJudge троши време и да је за даљу оптимизацију најважнији простор у парсирању и изградњи семантичког модела са CFG-ом.

Глава 10

Закључак

У закључку дајте кратак преглед онога шта урађено, са освртом на проблеме који су решени, предности и мане решења и правце даљег развоја.

Списак слика

Слика 1 Поједностављени AST за функцију из листинга 1.	10
Слика 2 CFG за функцију из листинга 1.	13
Слика 3 Главни ток анализе у алату RustyJudge.	21
Слика 4 Улазне тачке које користе заједничко RustyJudge језгро.	22
Слика 5 Ток изградње CFG-а унутар src/cfg модула.	28
Слика 6 Циљеви које CfgBuilder користи за break и continue у петљи.	31
Слика 7 Кораци које RustyJudge извршава при рачунању живости.	36

Списак листинга

Листинг 1	Једноставан JavaScript пример који се користи у овом поглављу.	9
Листинг 2	Псеудокод алгоритма за израчунавање достижних блокова.	14
Листинг 3	Псеудокод итеративне анализе живости.	16
Листинг 4	Интерфејс који имплементира свако правило.	23
Листинг 5	Скраћени приказ основних CFG структура.	29
Листинг 6	Скраћени приказ структуре LivenessResult.	36
Листинг 7	Пример у ком је прва вредност променљиве result мртва.	39
Листинг 8	Врсте исказа које правило no-dead-stores посматра као упис вредности.	44
Листинг 9	Пример у ком је прва додела променљивој x мртва.	44
Листинг 10	Облик CLI резултата.	47

Списак табела

Табела 1	Поређење алата релевантних за рад.	6
Табела 2	Нивои информација потребни за различите врсте lint правила.	7
Табела 3	Скупови анализе живости за CFG са слике 2	17
Табела 4	Главни делови RustyJudge пројекта.	20
Табела 5	Подаци које RustyJudge чува после анализе живости.	37
Табела 6	Живост на нивоу исказа за пример из листинга 7	40
Табела 7	Правила и подаци који су им потребни.	42
Табела 8	Резултати мерења над генерисаним корпусом.	51

Списак коришћених скраћеница

Скраћеница	Опис
API	Application Programming Interface (апликациони програмски интерфејс)
AWS	Amazon Web Services (Амазон веб сервиси)
CI/CD	Continuous Integration / Continuous Delivery (континуирана интеграција / континуирана испорука)
CORS	Cross-Origin Resource Sharing (размена ресурса између извора и дестинације различитог порекла)
CSS	Cascading Style Sheets (језик за описивање стилова)
DOM	Document Object Model (објектни модел документа)
DTO	Data Transfer Object (објекат за пренос података)
HTTP	HyperText Transfer Protocol (протокол за пренос хипертекста)
JSON	JavaScript Object Notation (формат за размену података)
JWT	JSON Web Token (сигурносни токен заснован на JSON формату)
RLS	Row-Level Security (сигурност на нивоу реда)
REST	Representational State Transfer (скуп правила за комуникацију између клијента и сервера)
RPC	Remote Procedure Call (позив удаљене процедуре)
SQL	Structured Query Language (структурирани упитни језик)
TLS	Transport Layer Security (безбедност транспортног слоја)
UML	Unified Modeling Language (језик за моделовање дијаграма)
URL	Uniform Resource Locator (јединствени идентификатор и локатор ресурса)
UI	User Interface (кориснички интерфејс)
UUID	Universally Unique Identifier (универзално јединствени идентификатор)
WAL	Write-Ahead Logging (записивање операција унапред)

Списак коришћених појмова

Појам	Објашњење
Асинхрони рад	Рад који се изводи независно од главног тока извршавања, омогућавајући наставак других операција без чекања на његов завршетак.
Bucket (S3)	Логичка јединица за складиштење у AWS S3 сервису, која организује фајлове у облаку.
Read-Only	Режим рада у коме су подаци само за читање, без могућности измене.
Cross-platform	Способност софтвера да се извршава на више различитих оперативних система из истог кода.
Connection pool	Механизам за управљање и поновну употребу веза са базом података како би се побољшале перформансе апликације.

Биографија

Овде написати своју кратку биографију.

Литература

- [1] F. Nielson, H. R. Nielson, and C. Hankin, *Principles of Program Analysis*. Springer, 1999.
- [2] Ecma International, “ECMA-262 ECMAScript Language Specification.” Accessed: June 28, 2026. [Online]. Доступно на <https://tc39.es/ecma262/>
- [3] OpenJS Foundation and ESLint contributors, “Core Concepts - ESLint.” Accessed: June 28, 2026. [Online]. Доступно на <https://eslint.org/docs/latest/use/core-concepts/>
- [4] RSLint contributors, “RSLint - GitHub repository.” Accessed: June 28, 2026. [Online]. Доступно на <https://github.com/rslint/rslint>
- [5] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*. Pearson, 2006.